

Unity Lighting & Visual Polish in VRChat

By MyroP

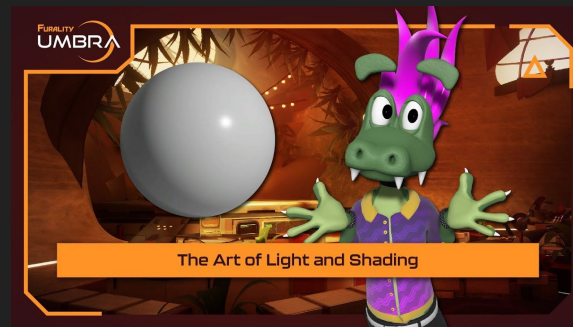
Who am I ?

- 33 years-old French software developer
- Creating VRChat worlds since 2021, as a hobby
- Some worlds I created : VRbomber, VRtual park, Nucleus etc.

Download the presentation



Shoutouts



<https://www.youtube.com/@Furality/videos>

Content

Level : Beginner

Focus : Realism, light baking,
shading.

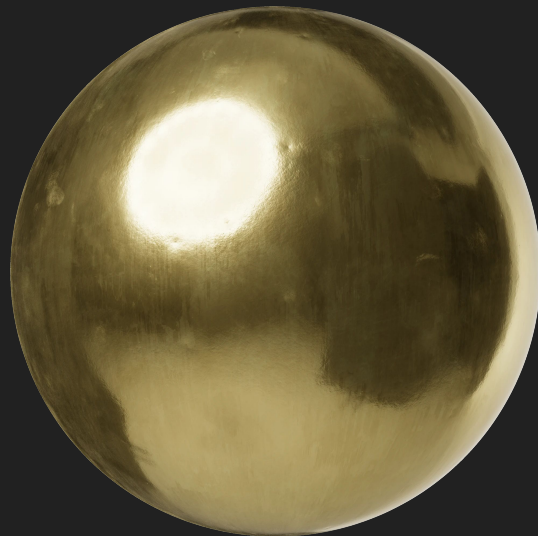


Physically based rendering/shading (PBR)

- Concept that uses realistic lighting and material data to mimic real-world surfaces.
- Goal : Achieve realism
- It's a concept, implementations may vary (e.g. Filament)

Unity provides a few PBR shaders:

- Standard Metallic
- Standard Specular

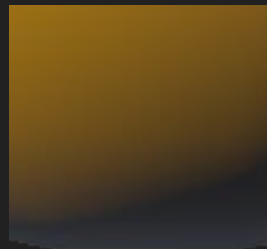
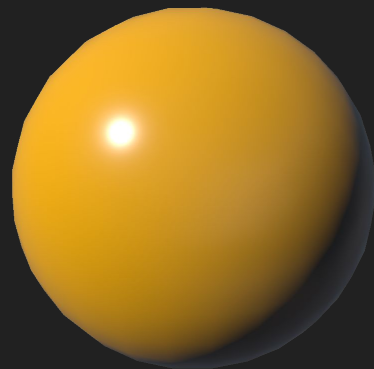


Physically based rendering/shading

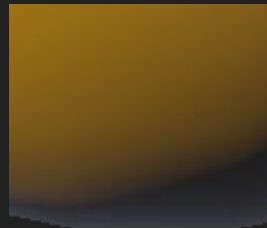
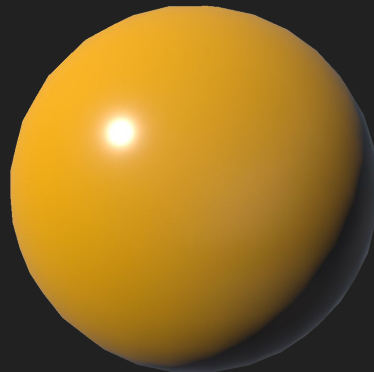
PBR defines how materials should behave physically, but not how that behavior must be implemented.

**Implementations may vary,
and render differently**

Google Filament (Silent
Filamented shader)



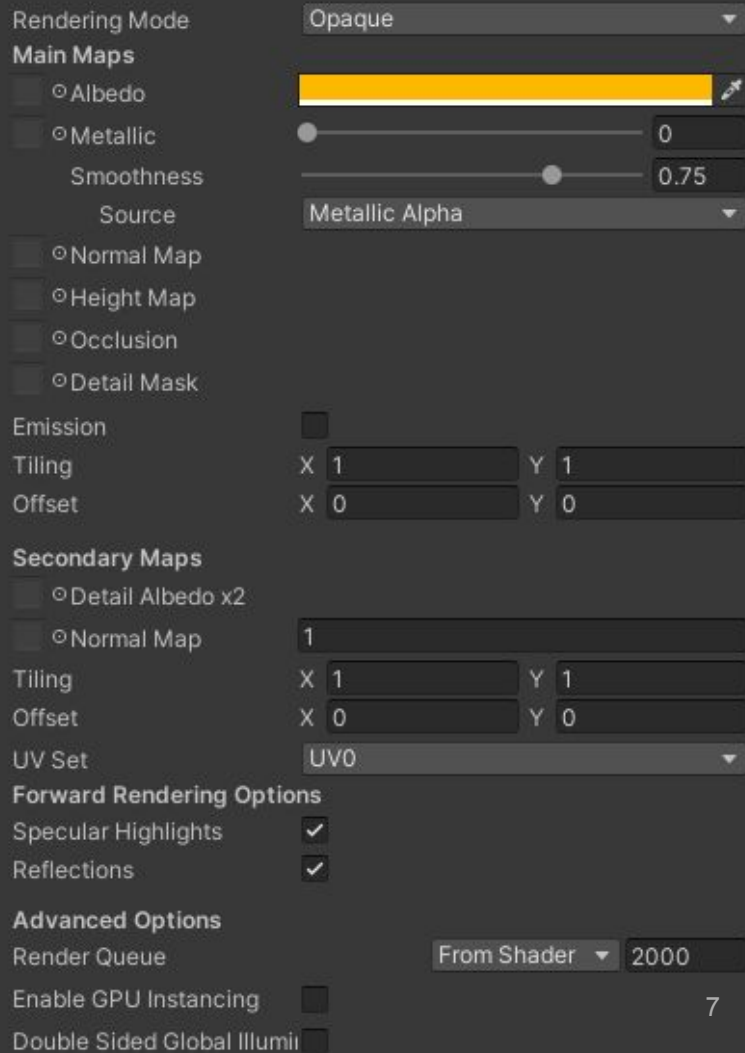
Standard Unity PBR



Unity Standard (Metallic)

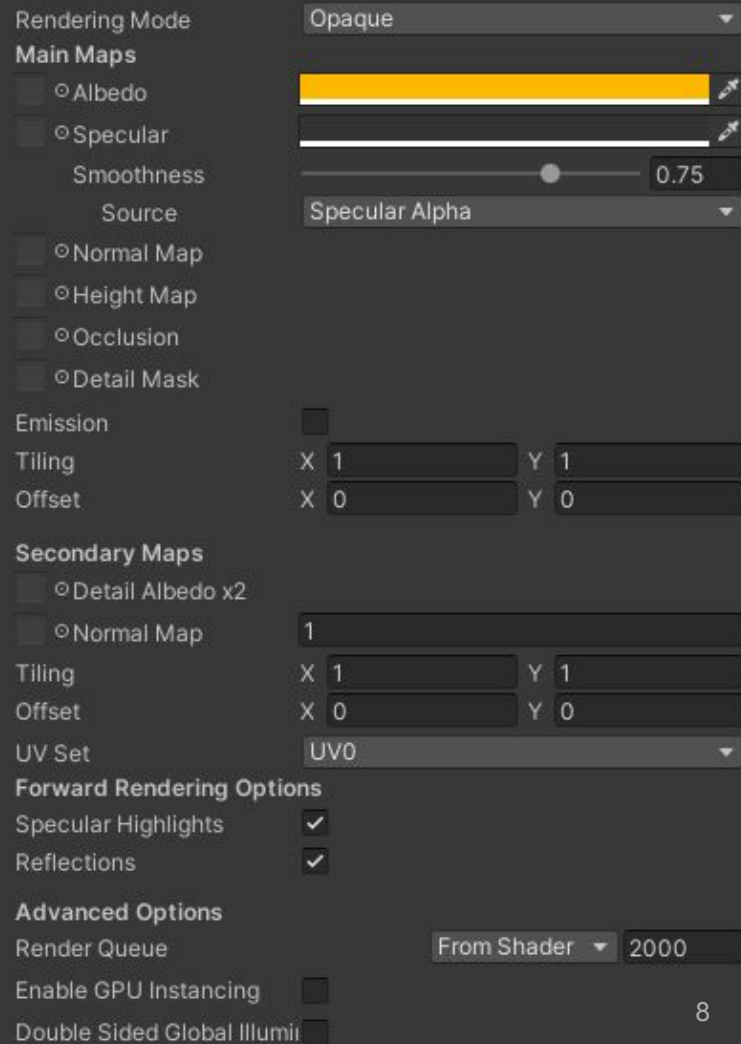
- Albedo : Main color
- Metallic/Smoothness :
 - Metallic : binary-ish metallic value (Red channel)
 - Smoothness : (Alpha channel)
- Normal/Height map
- Occlusion (Green channel)

Tip: If you don't have a metallic texture and the albedo is opaque, you can store the smoothness data in the albedo alpha and set "Source" to "Albedo Alpha".



Unity Standard (Specular)

Very similar to Standard metallic, but the metallic mask is replaced by an RGB specular texture



Specular vs. Metallic workflow

Metallic workflow :

- Most common
- Can simulate most surfaces
- Easy to use

Specular workflow:

- More control over reflections
- More artistic control

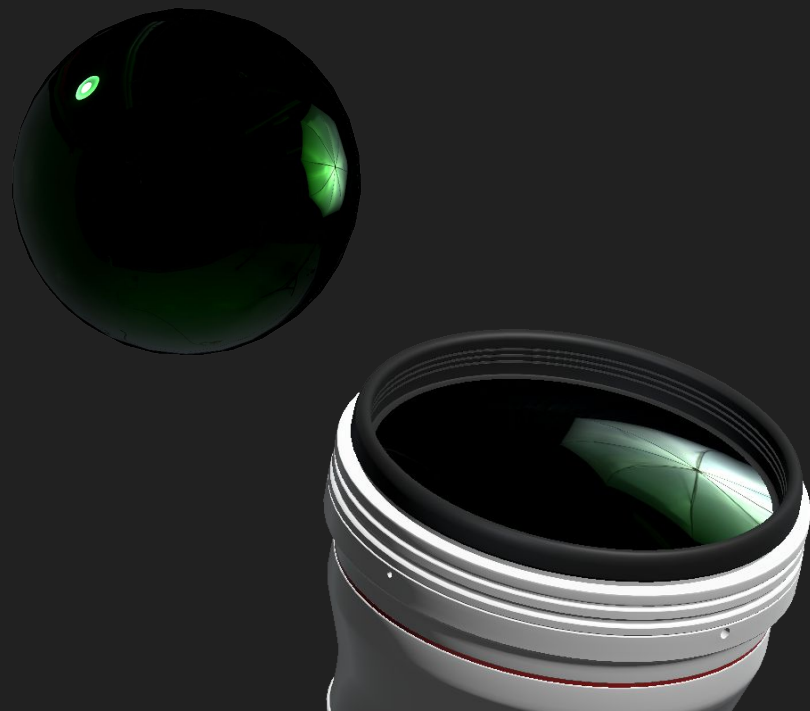
	Metallic workflow	Specular workflow
Metal vs non-metal handling	Automatic via metallic property (0 or 1)	Manual
Specular reflection color control	For metals : Driven by Albedo For non-metals : Neutral (light color)	Directly via Specular color
Specular intensity	Fixed specular for non-metals	Directly via Specular color

Specular workflow

Example : Coated camera lens

Specular reflection is colored, but the surface (glass) is not metallic, so we can use the specular workflow.

Specular color : dark green

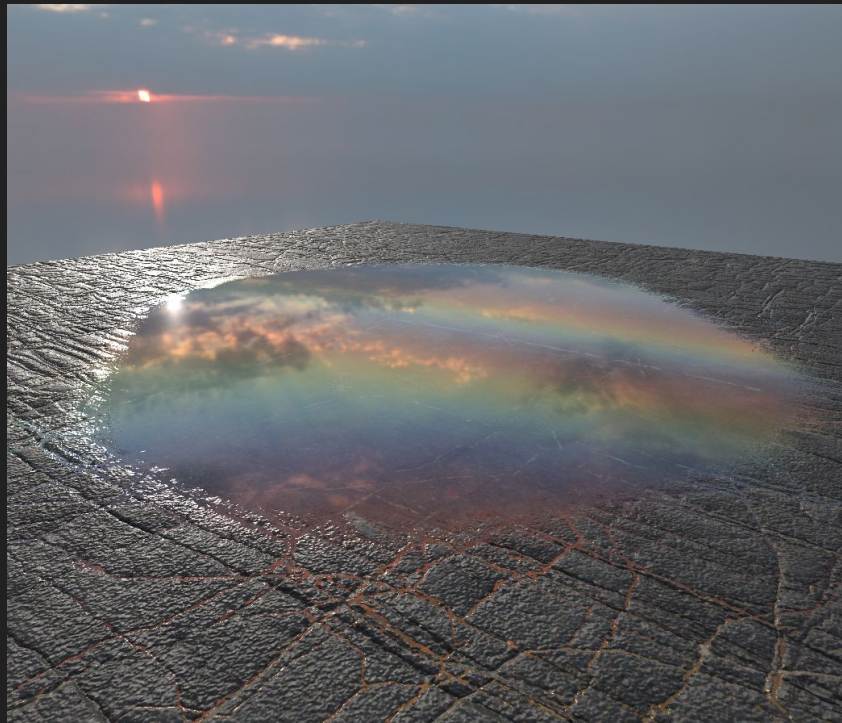
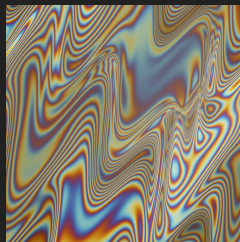


Specular workflow

Example : Oil puddle

Specular reflection is colored, but the surface (oil) is not metallic, so we can use the specular workflow.

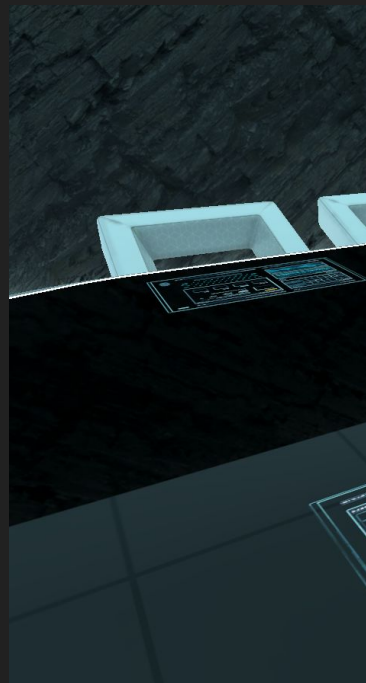
Specular texture :



Specular workflow

A darker specular color/value decreases specular intensity.

Specular 0.4



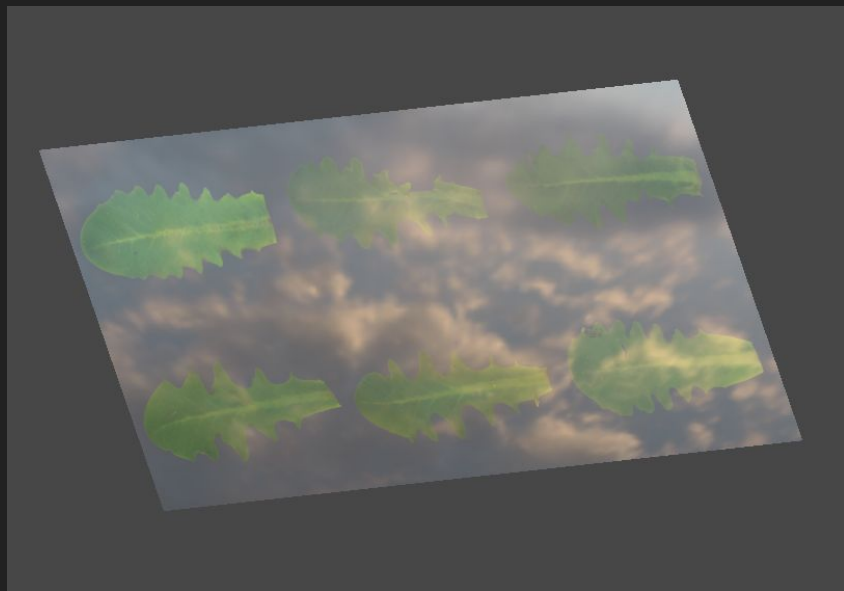
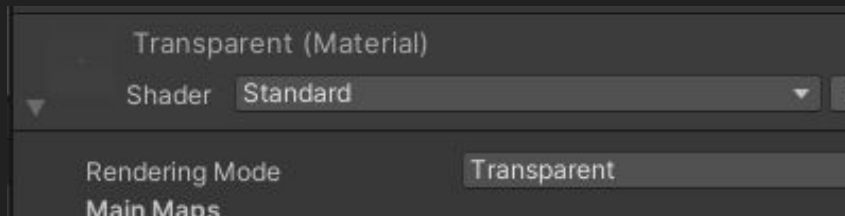
Specular 0.1



Rendering mode

Transparent: Opacity doesn't affect specular reflections

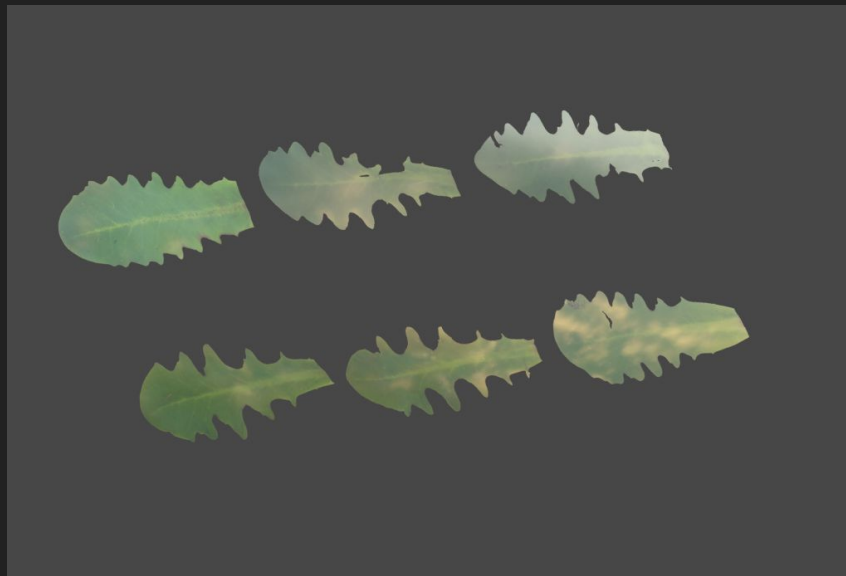
Usecase : Great for glass



Rendering mode

Fade: Opacity affects specular reflections, no visible reflections on fully transparent parts

Usecase : Anything that needs to be semi-transparent with a cutout effect : particles, explosions, smoke, trails etc.

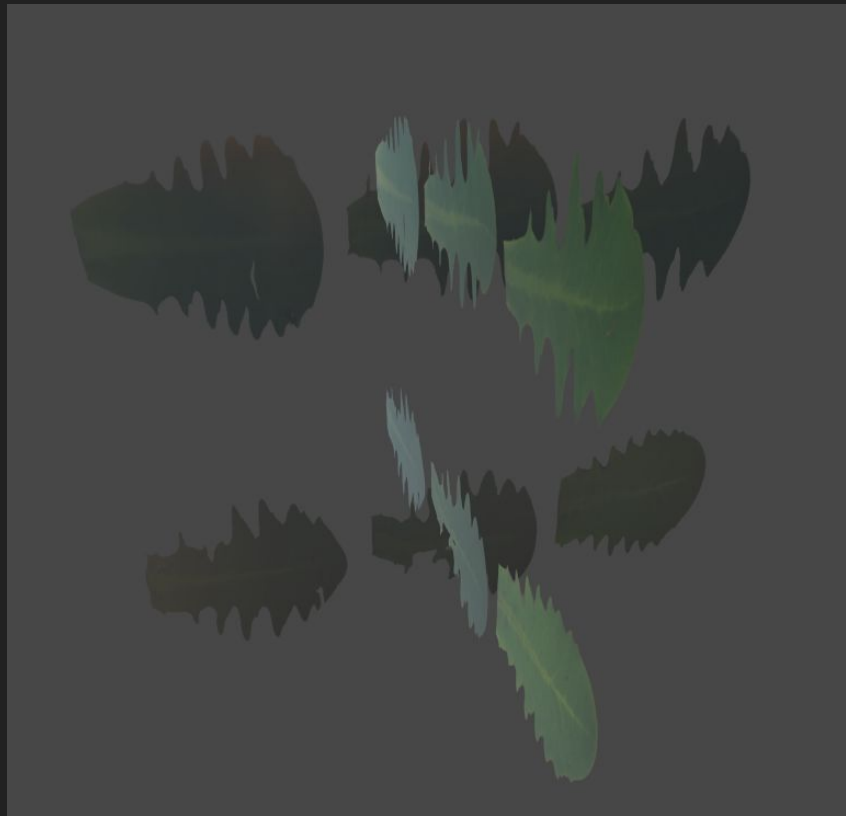


Rendering mode

Transparent and **Fade** can cause rendering issues caused by rendering order.

Fix : Difficult, playing around with the render queue can sometimes help. avoid using those rendering modes on dense foliage or tree leaves.

Example: Intersecting planes

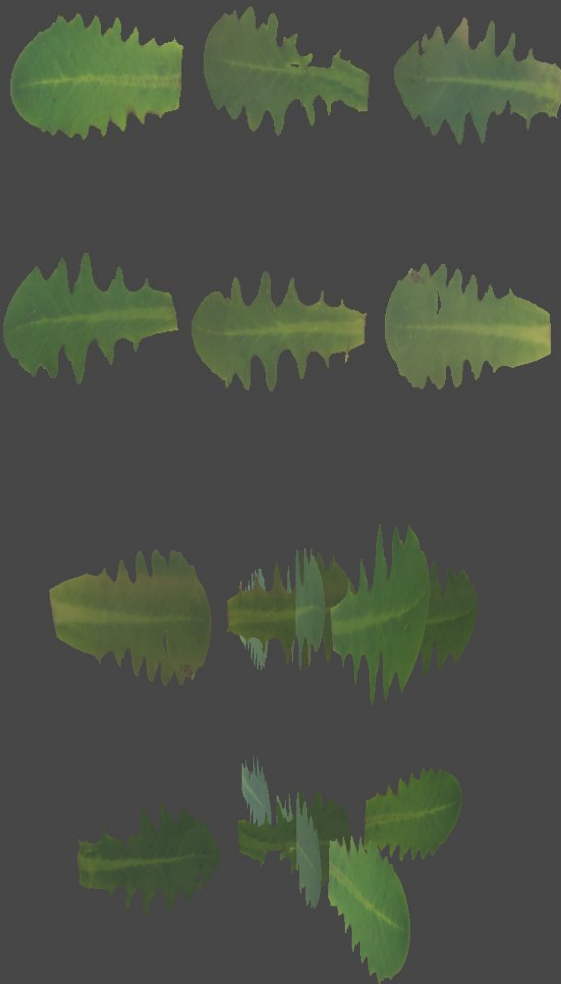


Rendering mode

Cutout: Technically opaque, pixel is either on or off

No rendering issues on intersecting objects.

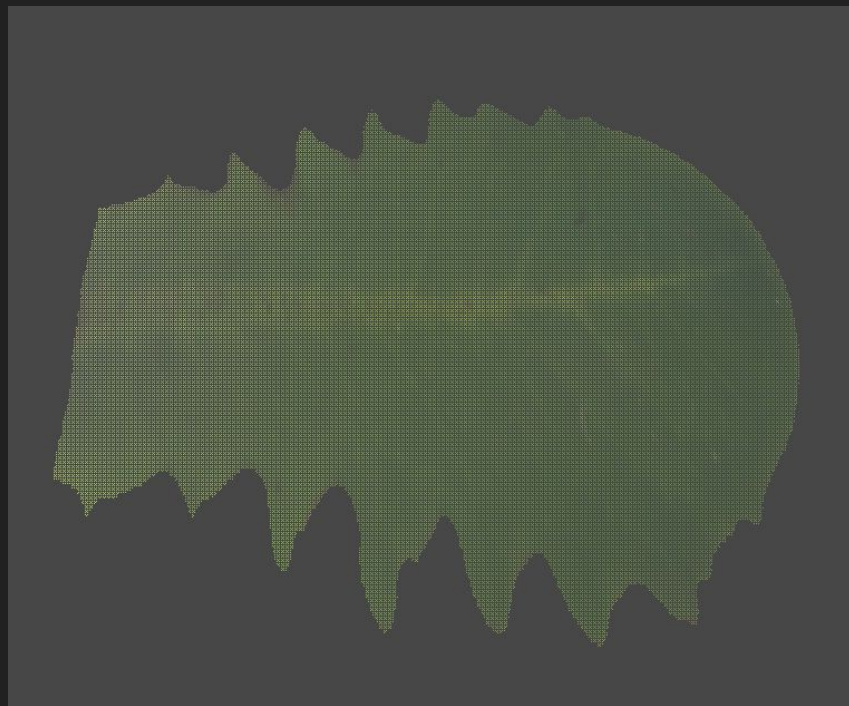
Usecase: Dense foliage, tree leaves...



Rendering mode

Dithering: Not supported by many shaders, cutout variant that alternates on/off pixels to simulate transparency

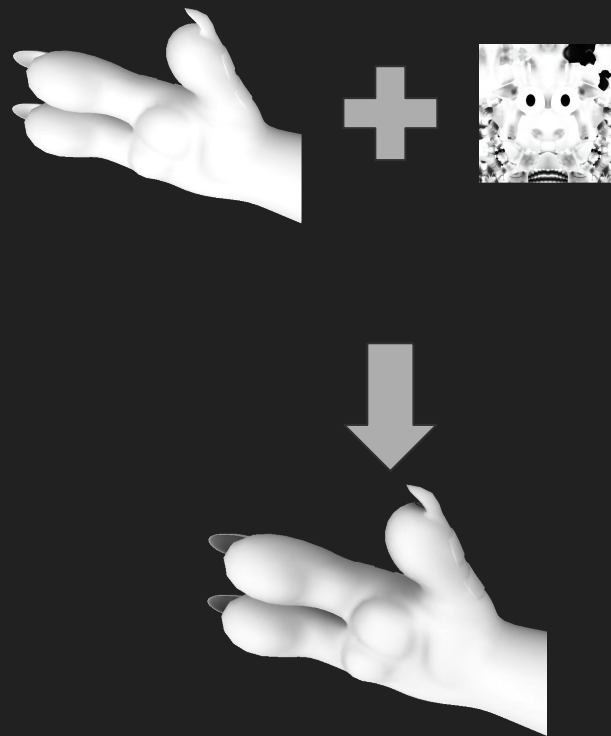
Interesting alternative to achieve transparency if you must use opaque shaders



Ambient occlusion (AO)

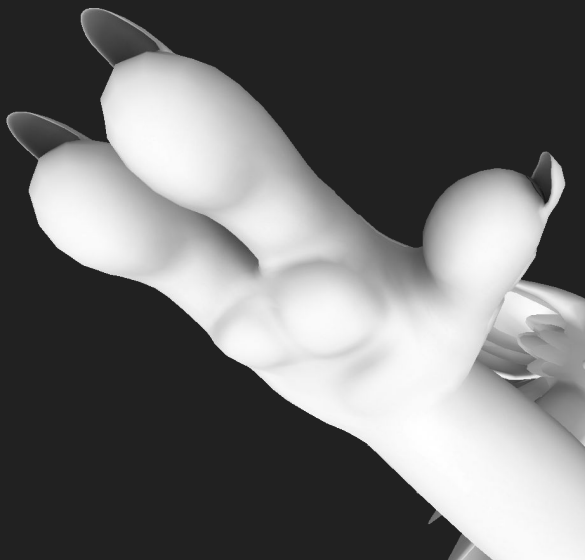
- Indirect lighting multiplier
 - 0 : No indirect lighting
 - 1 : Full indirect lighting
- Indirect light includes :
 - Ambient light = Light who doesn't come from any specific source object.
 - Environment reflections

Side note : AO is technically not PBR accurate

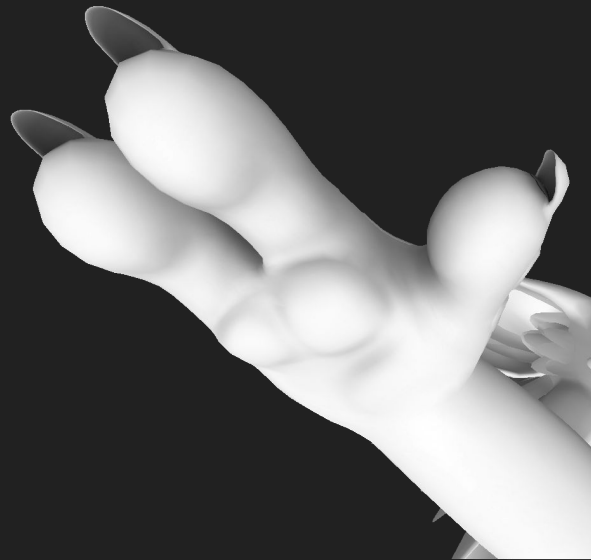


Q: Why not baking AO directly into the main texture?

AO



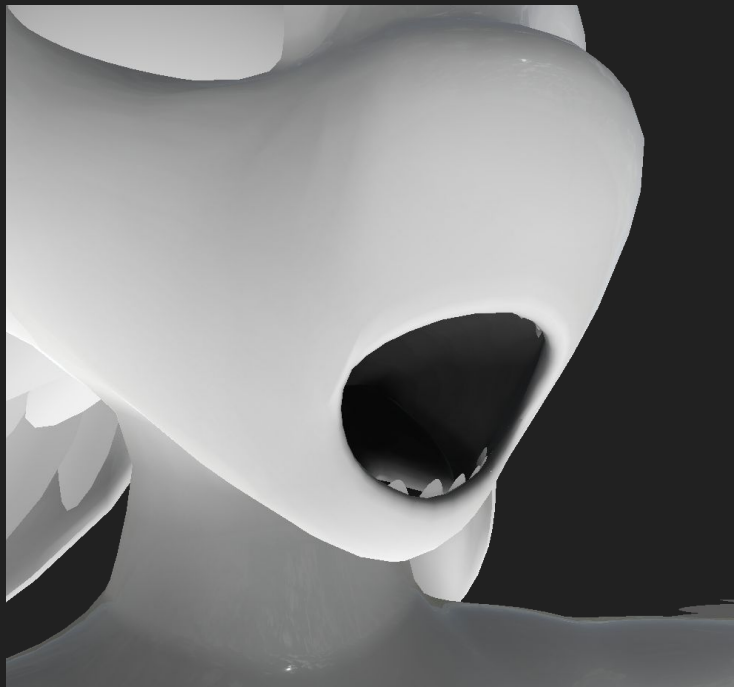
AO baked into main texture



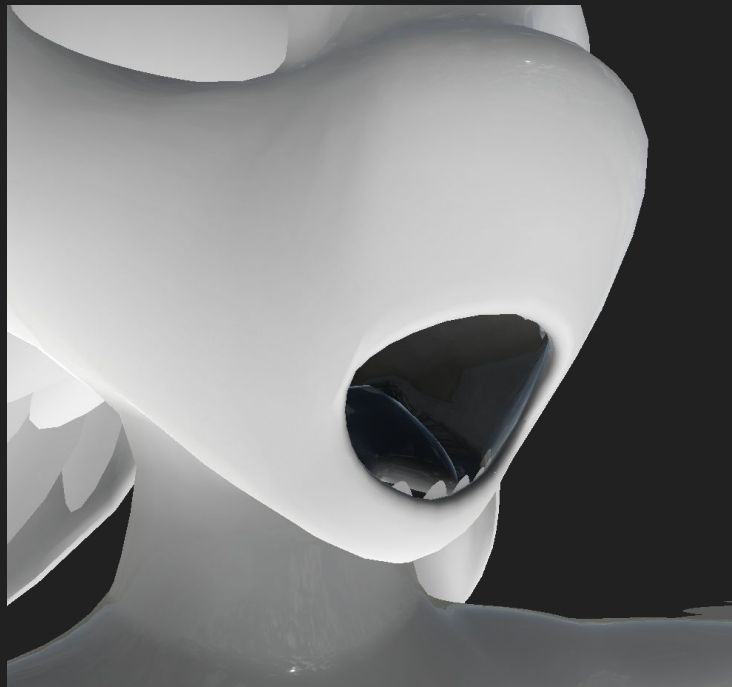
A : Because AO is an indirect light multiplier

AO can be used to remove reflections where we don't want them

AO



AO baked into main texture



A : Because direct lighting diminishes AO

AO



AO baked into main texture

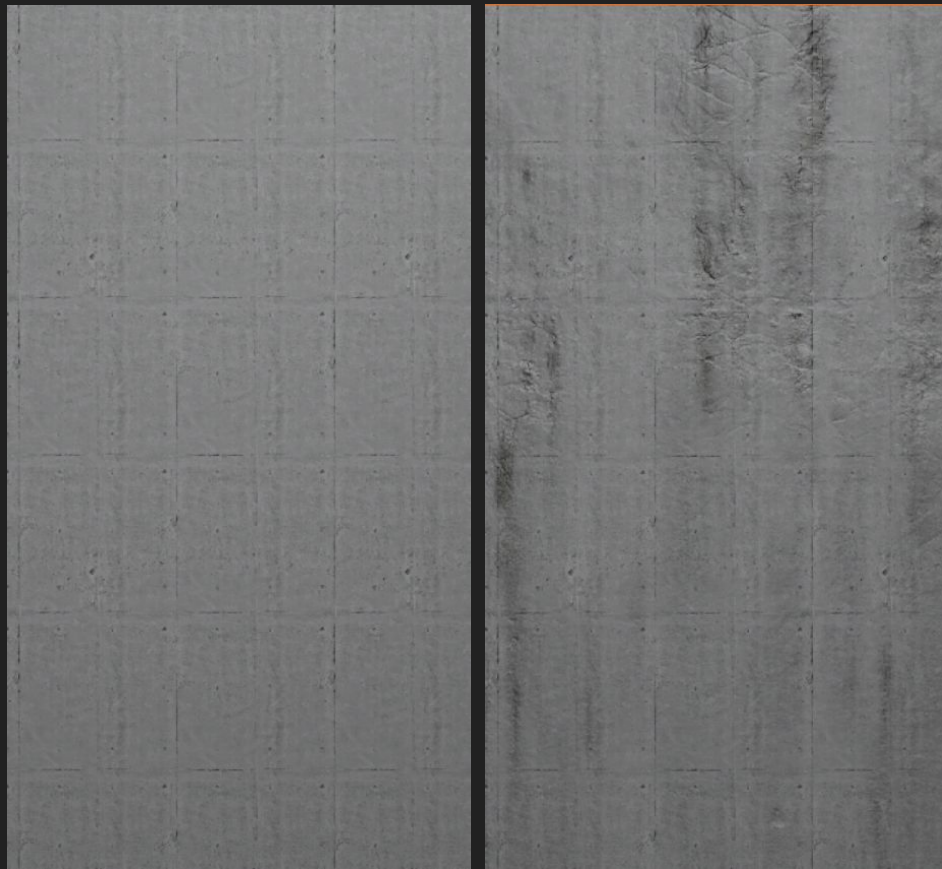


Details

Overlays a texture (usually tiled at a different scale) on top of a base map to :

- Enhance fine surface detail.
- Break repetition

Can also be used on avatars to add details (cloth, skin, fur etc.)



Where can we download free PBR Materials

AmbientCG : <https://ambientcg.com>

Polyhaven : <https://polyhaven.com>

Searching for “Textures CC0” gives many results



Crunch compression

Decreases built size, but can heavily affect the visuals of certain types of textures, like normal maps

No compression (5.3 MB*)

Normal quality (1.3 MB*)

Normal +crunch (200 kB*)

BC5** Format (1.3 MB*)



**MB size shown in Unity, it is not the VRAM usage.*

***BC5 is the recommended format for normal maps*

Lightmapping

Issues with RT lights

- RT lights are performance heavy
- RT lights don't handle light bounces → less realistic lighting
- Limited number of lights rendered at once (defined in the project settings)
 - VRChat : Players can turn OFF “pixel lights”, 8 max



Solution : Lightmapping

- Precomputes lighting → better performance
- Produces softer, more realistic lighting (handles light bounces...)



Lightmapping : How it works

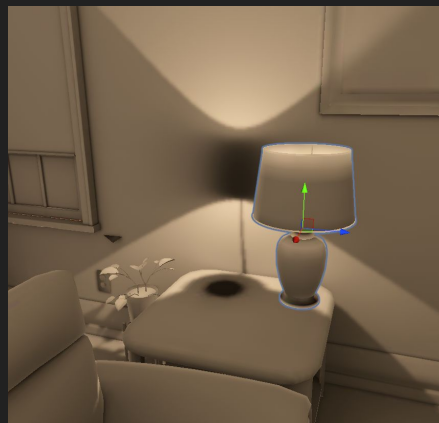
- Lighting is baked and stored in a lightmap texture
- Each object occupies a unique area in the lightmap
 - Uses the mesh's second UV channel (UV1), UVs must not overlap
- Lightmap applied to objects at runtime

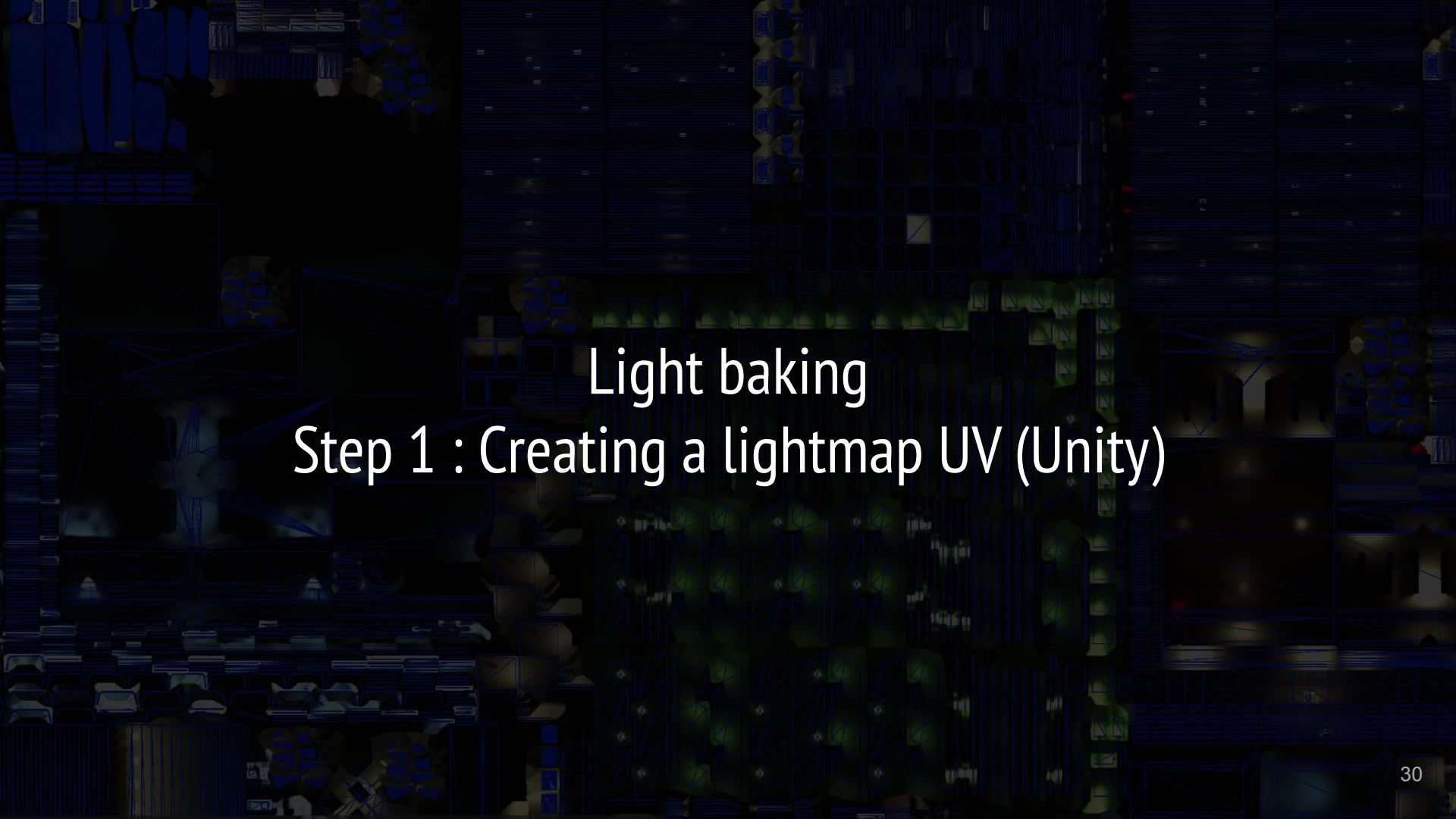
No RT lighting calculations needed



Lightmapping drawbacks

- Generates a lightmap texture
 - Increases build size
 - Uses more memory
- Not (really) supported for dynamic or moving objects
- Not (really) suitable for changing lighting condition.
 - Day/night cycle, light toggles etc.
- Baking can take quite some time, good graphic card required.
 - More VRAM = better
- Limited texture resolution → Not great for huge open worlds



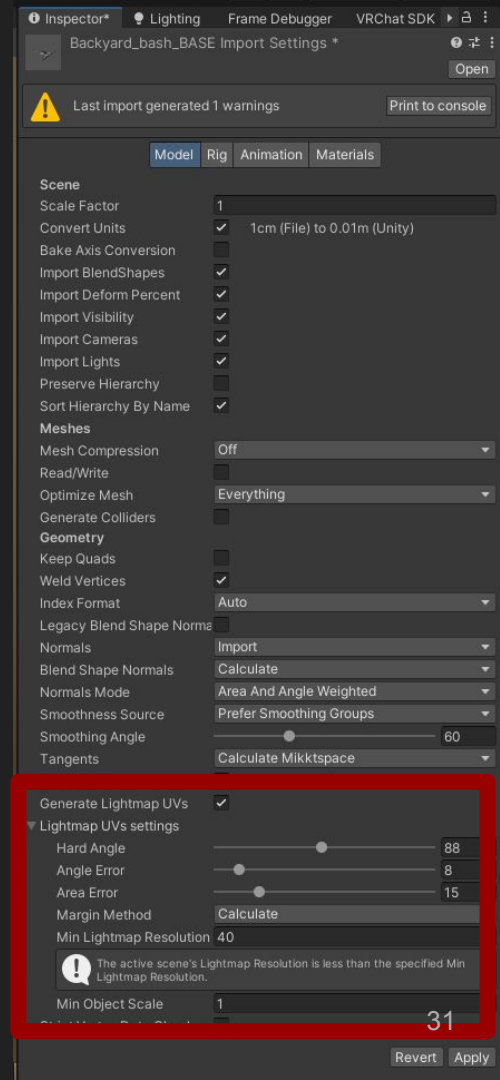


Light baking

Step 1 : Creating a lightmap UV (Unity)

Step 1 : Creating a lightmap UV (Unity)

- Project window → Select the mesh
- In the inspector, check “Generate Lightmap UVs”
- Generated lightmap UV gets stored in UV1

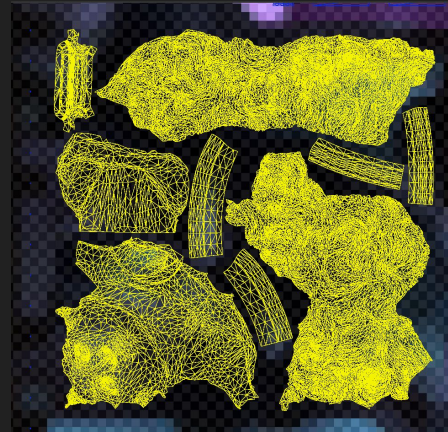


Step 1 : Creating a lightmap UV (Unity)

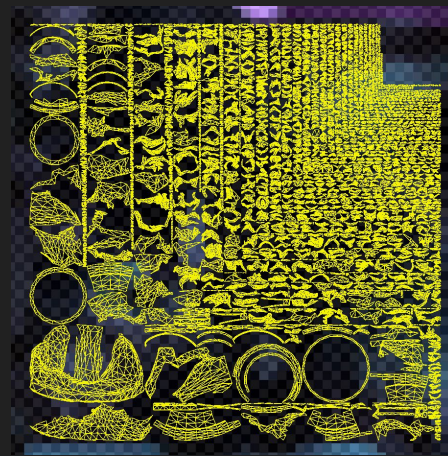
If UV1 is missing, UV0 is used instead.

This works if:

- No overlapping UVs
- Enough padding between UV islands
- Good enough UV1 (good placed seams, minimum stretch...)



"Generate Lightmap UV" unchecked



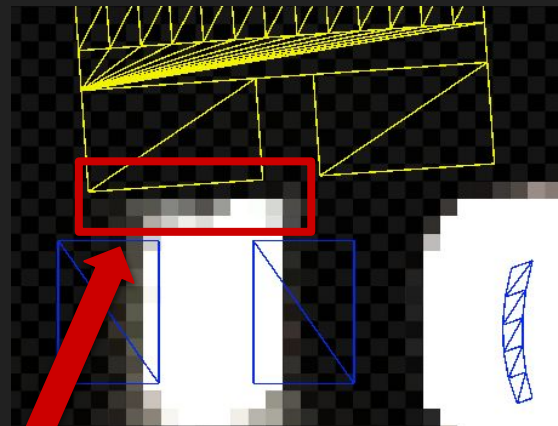
"Generate Lightmap UV" checked

Step 1 : Creating a lightmap UV (Unity)

Lightmap bleeding: Baked light from one UV island bleeds into another island

Fix:

- Increase padding.
- Group brighter islands together. (Blender...)



Creating a lightmap UV (Blender)

Advantages :

- More control over density
- Faster Unity import time

Drawbacks:

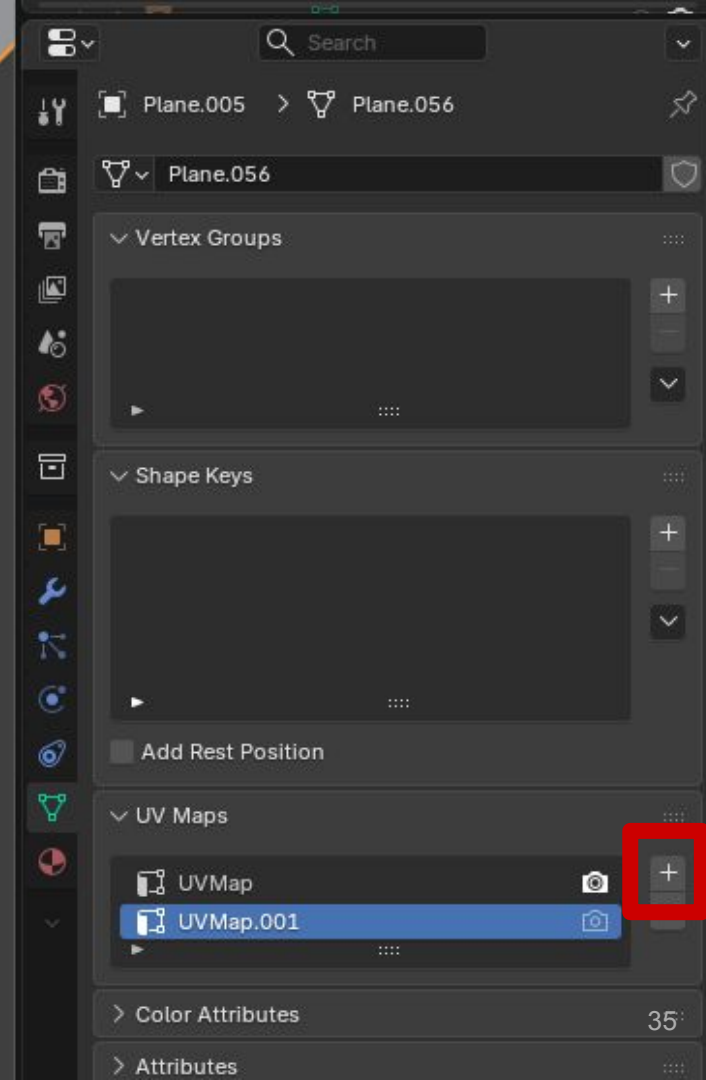
- More manual work
- No automatic padding control



Creating a lightmap UV (Blender)

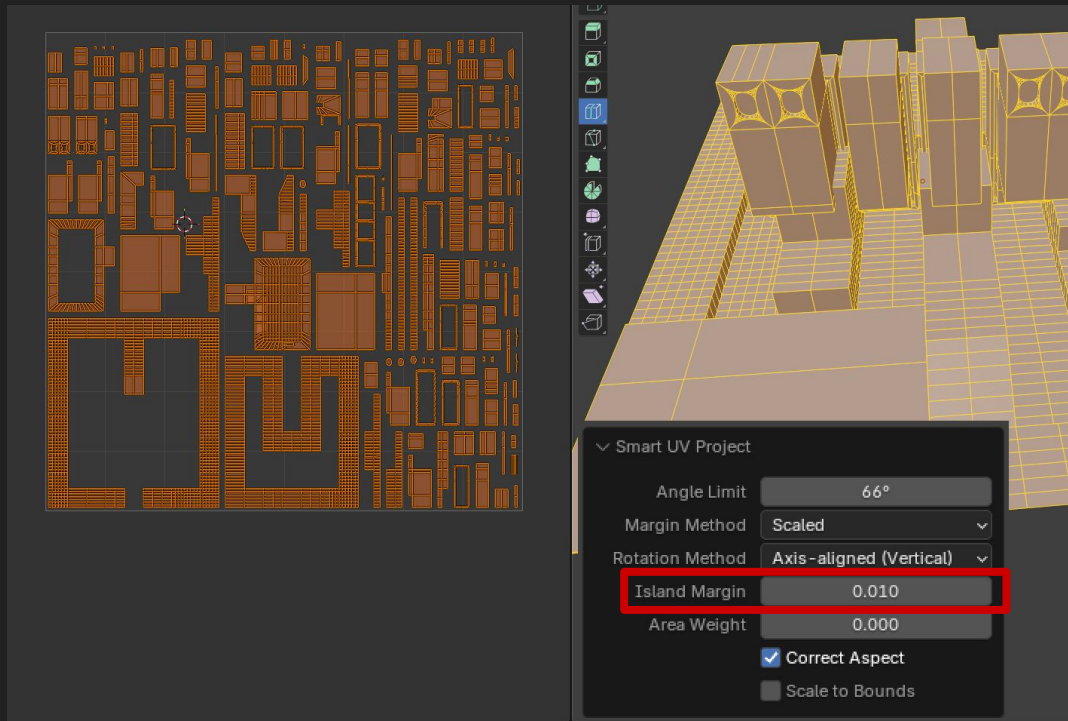
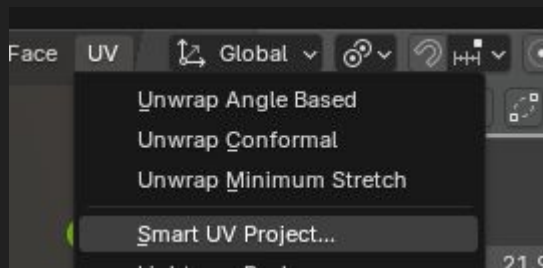
- Add a new UV map (name doesn't matter)
- Unwrap on the new UV
 - Manually (Mark seams...)
 - Automatic (built-in tools, add-ons...)

UV islands should not overlap, enough padding



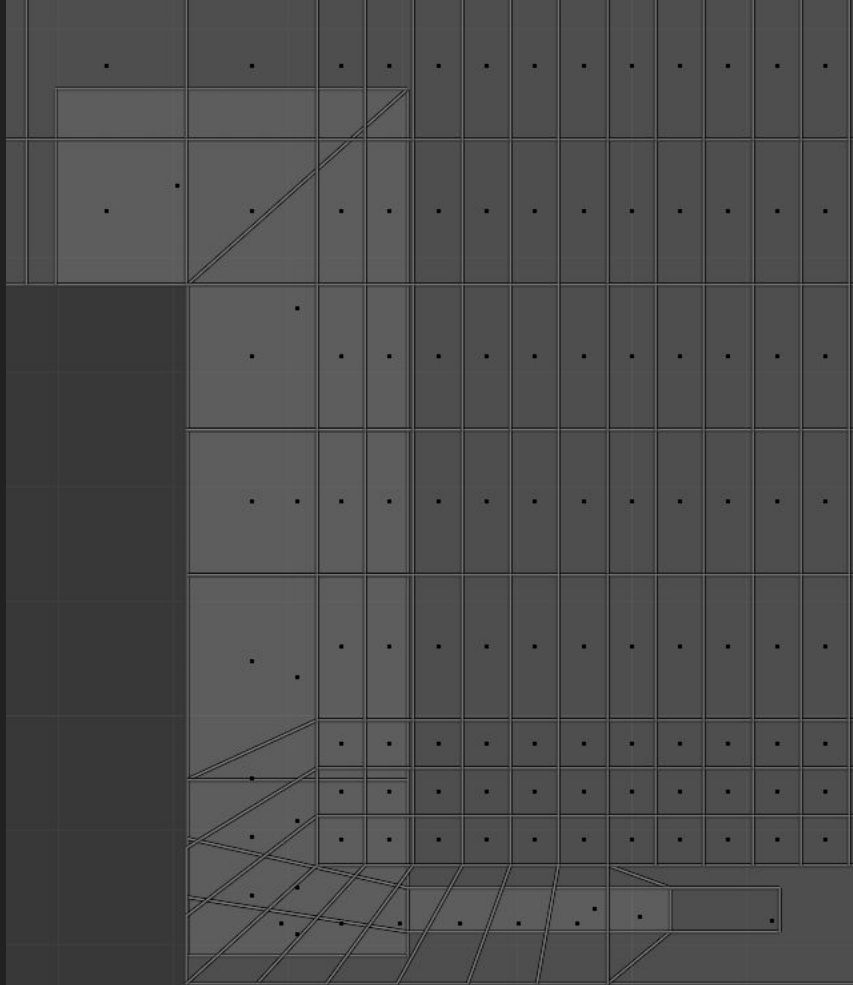
Creating a lightmap UV (Blender)

- “Smart UV Project...” is quick and easy
- “Island margin” to set the margin, increase that value to prevent lightmap bleeding



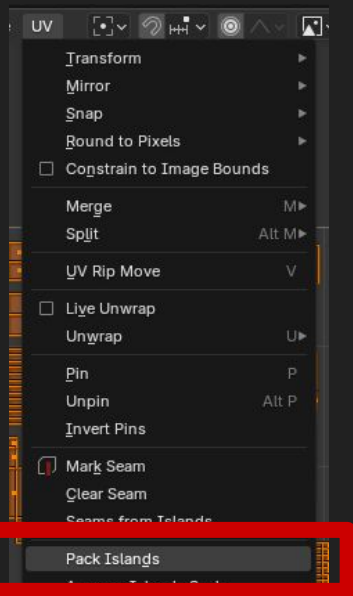
Creating a lightmap UV (Blender)

- “Smart UV Project” creates sometimes overlapping faces, in that case you should manual unwrap affected areas.

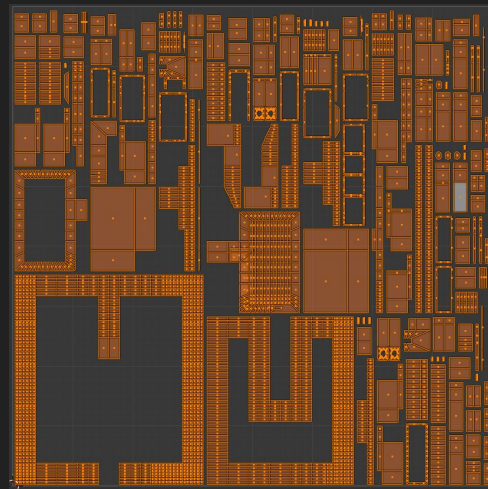


Creating a lightmap UV (Blender)

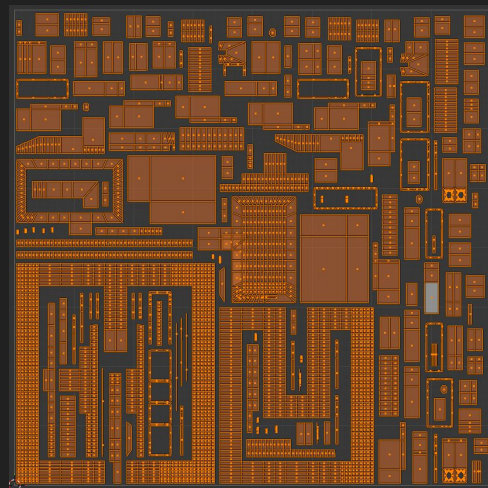
“Pack Islands” option to fill holes



Before pack



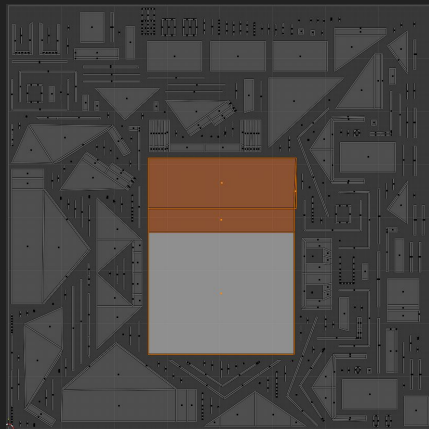
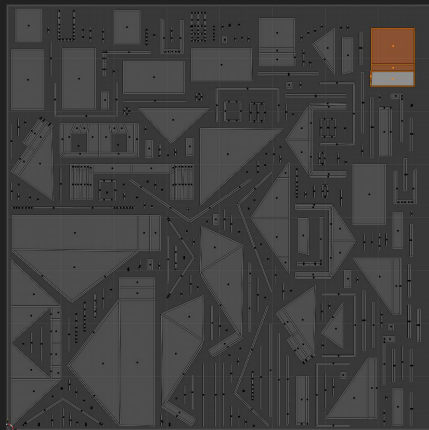
After pack



Creating a lightmap UV (Blender)

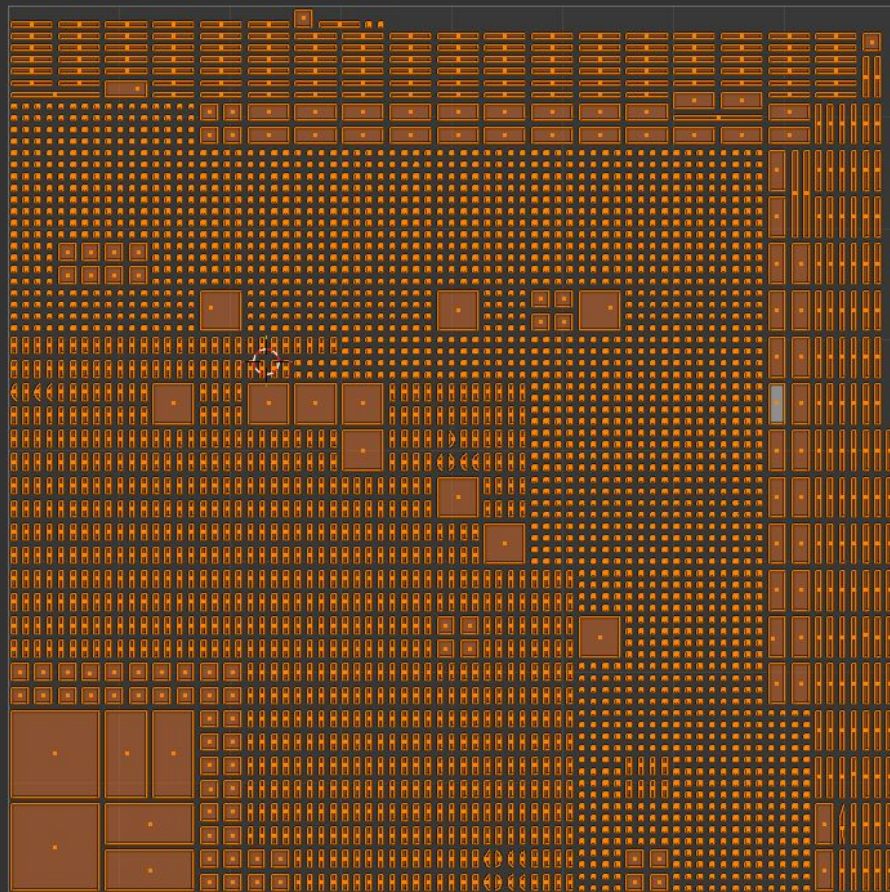
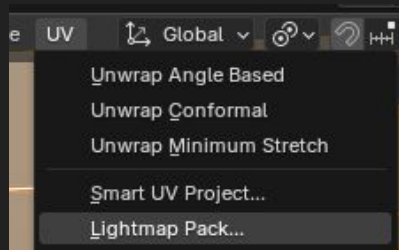
Customizable lightmap density per island
to add more details on certain islands
(decreases quality on other islands)

- Select island (L key)
- Scale (S key)
- UV → Pack islands
 - Don't forget to add some padding



Creating a lightmap UV (Blender)

- Do not use the “Lightmap pack...” option :
 - It puts each face on a separate island → Not space efficient, decreases baking quality.
 - Stretches texels

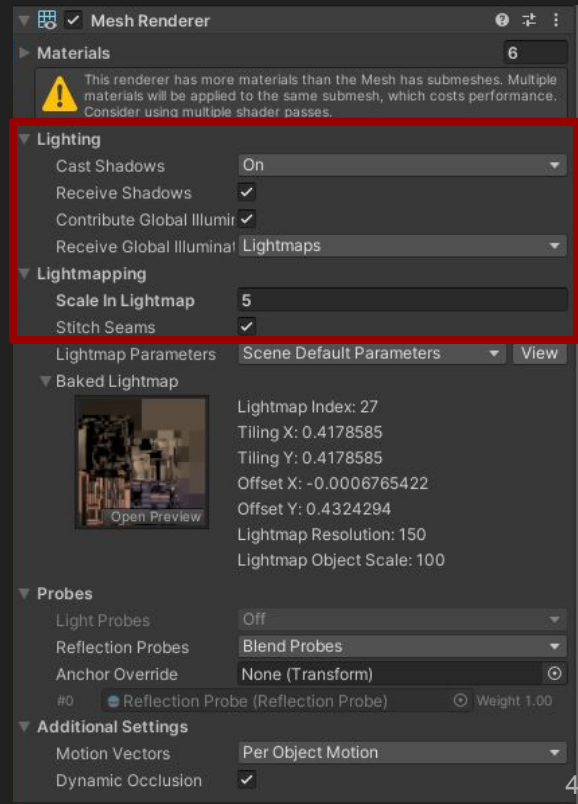
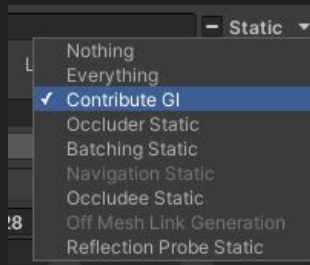


Light baking
Step 2 : Preparing the scene

Step 2 : Preparing the scene

Mesh Renderer settings :

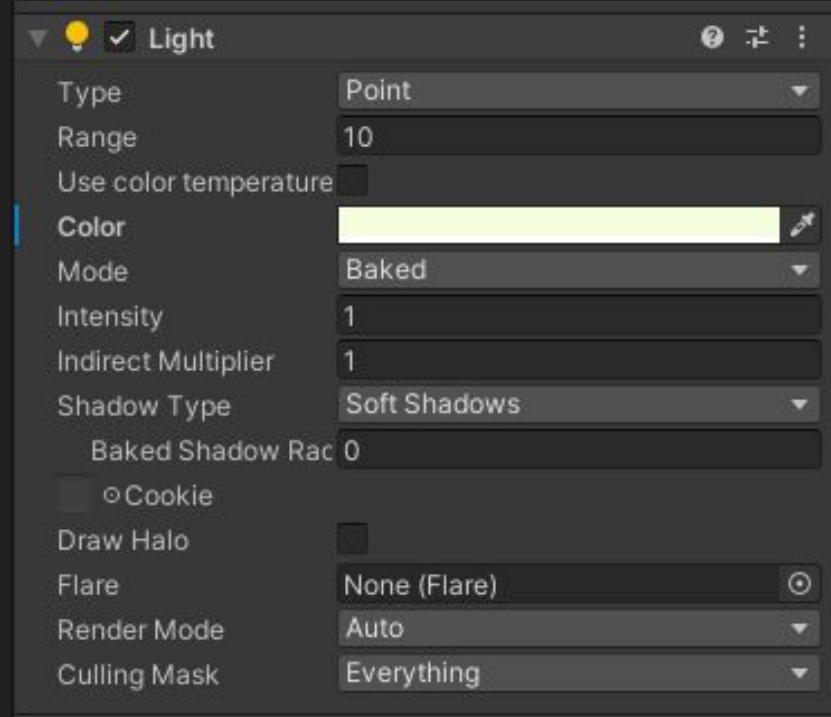
- Mark object as “Contribute GI”
- Cast/Receive Shadows **On**
- Receive GI **Lightmap** or **light probes**
(For small objects that are difficult to lightmap)
- “Scale In lightmap” to customize resolution



Preparing the scene : Direct lights

Light settings

- **Mode : Baked**
- **Shadow type : Soft/hard shadow**
- Type :
 - Directional Light (e.g., sun)
 - Point Light (e.g., a light bulb)
 - Spot Light (e.g., flashlight, stage spotlight)
 - Area Light (LED strip, TV screen)

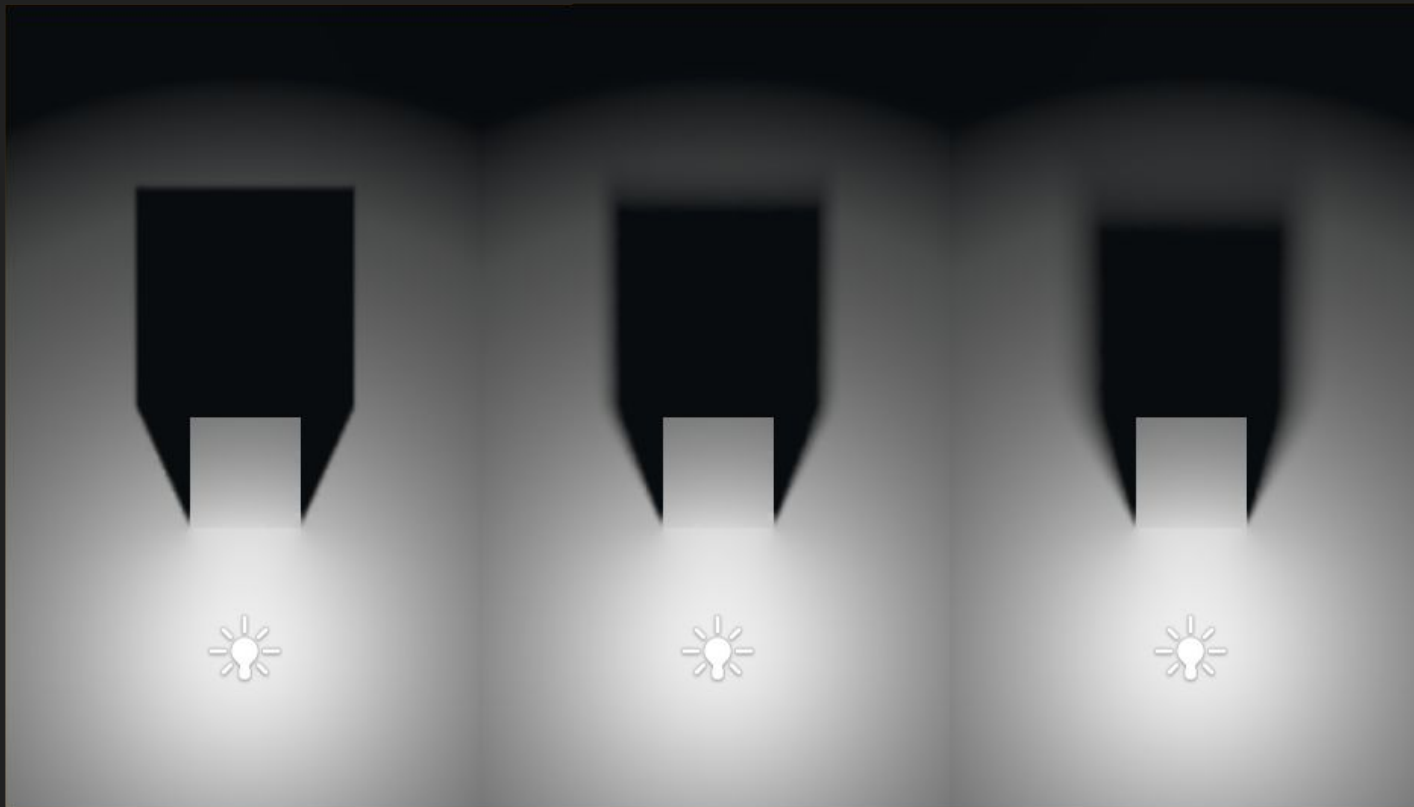


Baked Shadow Radius (Soft shadow) $\approx$ Radius of light source

0

0.05

0.1



Indirect multiplier (changes intensity of bounced light)



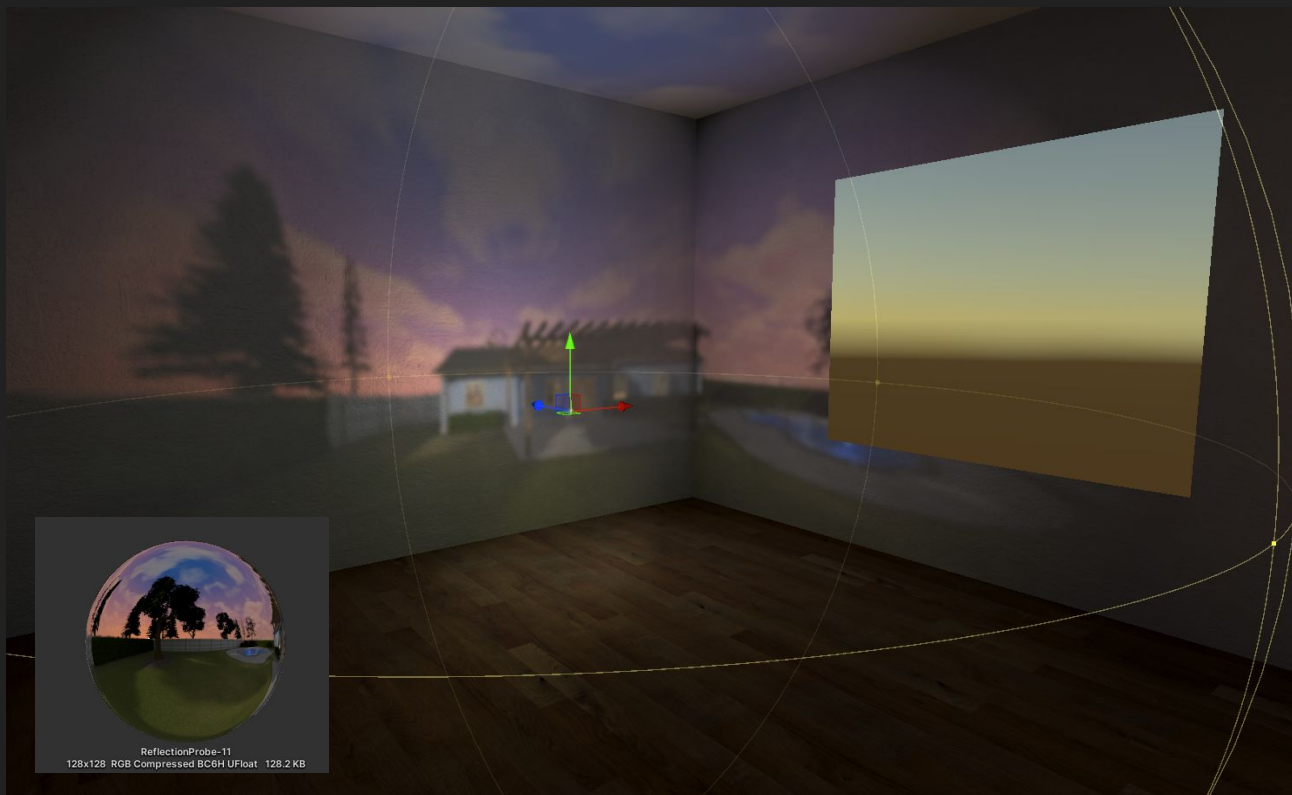
Cookie texture

Spotlight : Customize its shape



Cookie texture

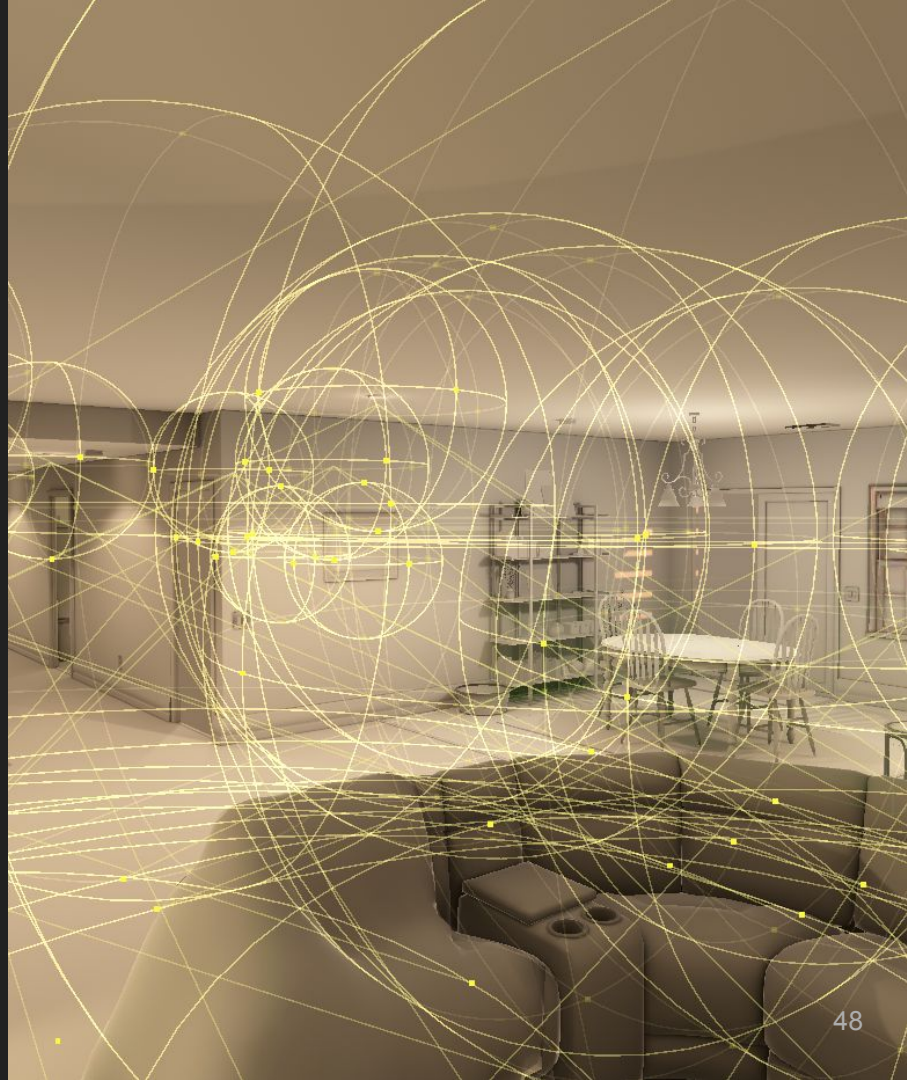
Point Light : Project a cubemap into the environment using a cubemap as cookie texture



Preparing the scene : Direct lights

Add as many light sources as you need.
They're only used for rendering, so they have no performance impact.

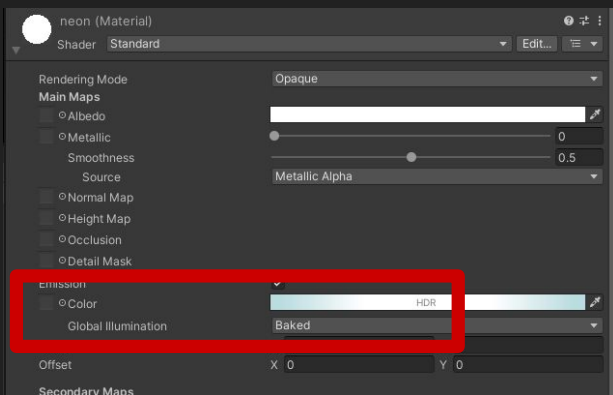
Example : Spookyghostboo's "backyard bash" world



Preparing the scene : Emissive lights

Emissive lights (material) can also be baked, but they usually give noisier results. HDR controls intensity.

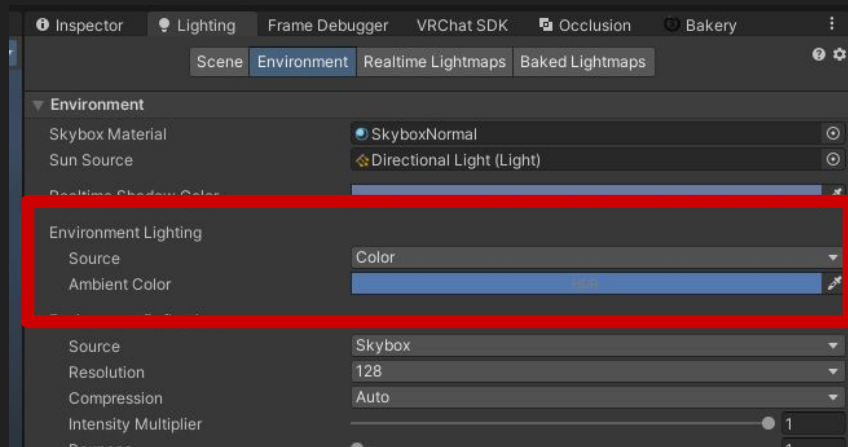
Setup : “GI” to Baked in the material settings, object must be static (contribute GI).



Preparing the scene : Environment lighting

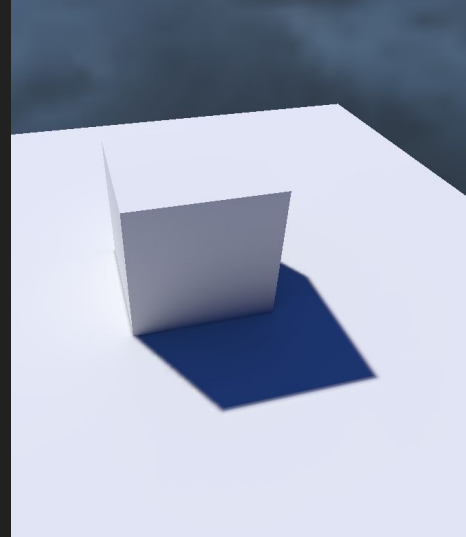
Environment lighting : Color

Interesting way to tint shadows or to make them brighter when light bounce isn't enough



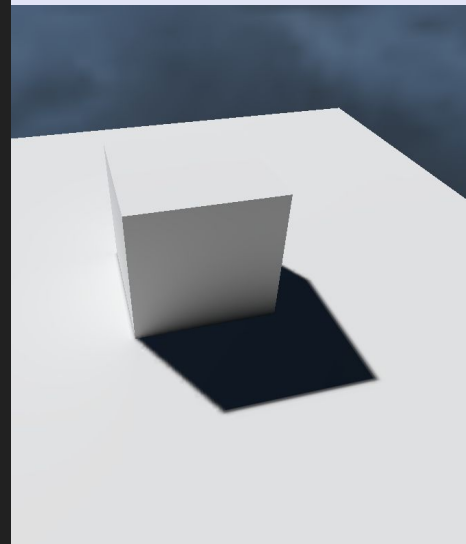
Ambient color : Blue

with white Dir. light



Ambient color : None

with white Dir. light

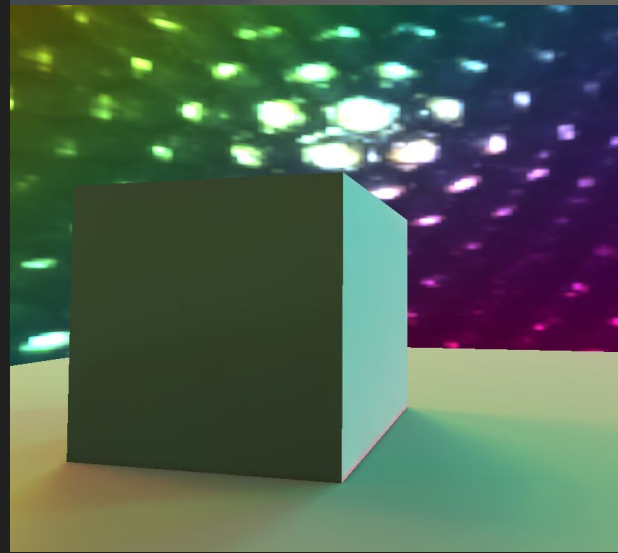
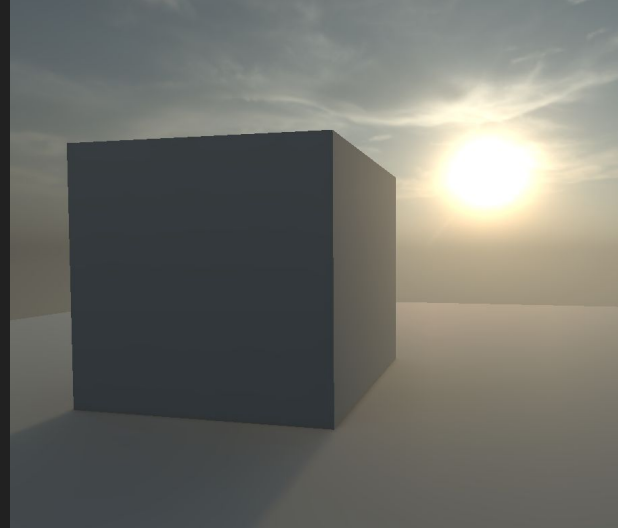


Preparing the scene : Environment lighting

Environment lighting : Skybox

Light emitted from skybox, supports HDR, brighter spots will emit more light :

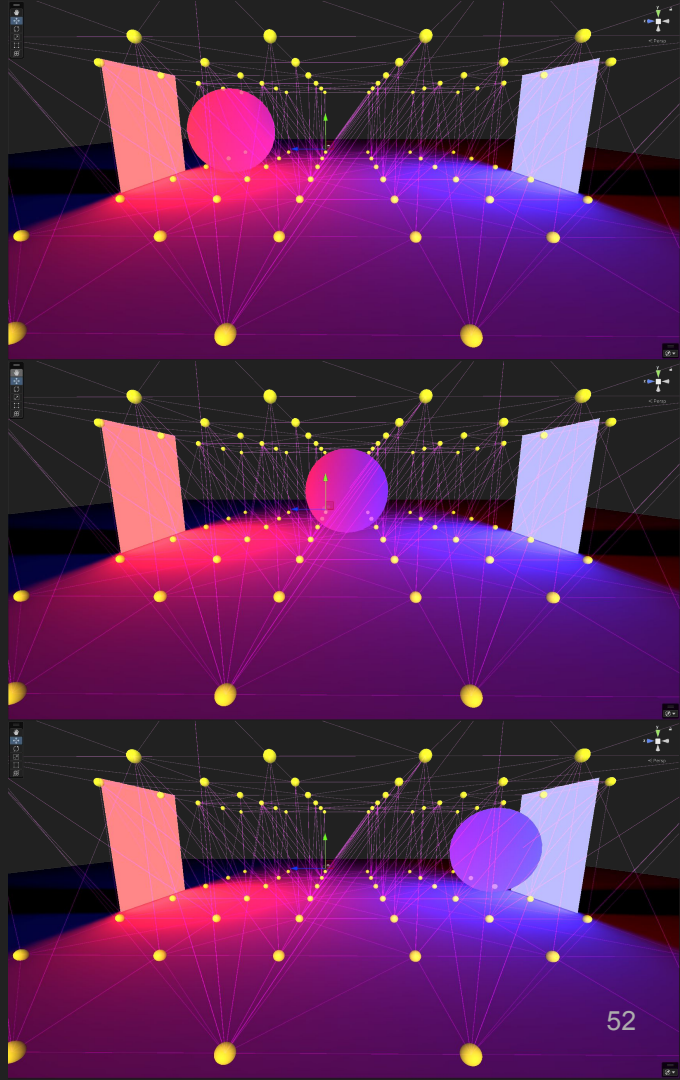
- Skybox has a sun? The sun will cast a shadow without needing a directional light
- Skybox is blurry/cloudy/messy? Light may come from all directions, making shadows softer. A very colorful skybox will emit very colorful lights.



Preparing the scene : Light probes

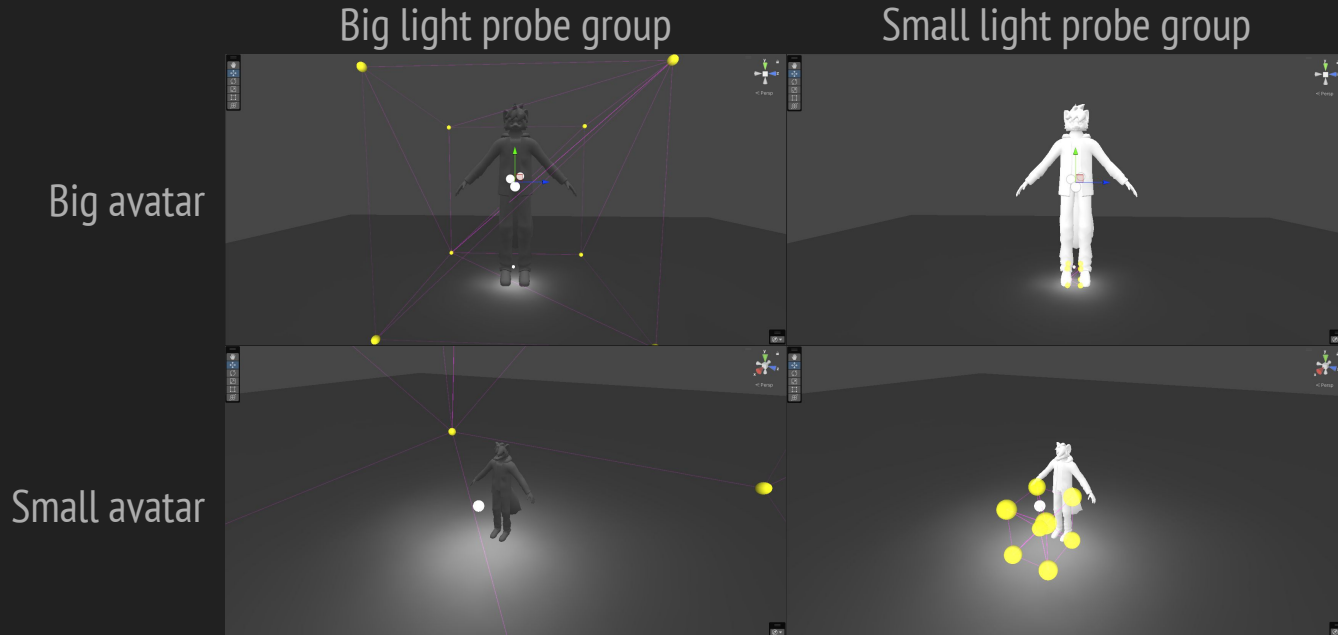
Lighting up avatar with light probes:

- Basically “Light baking at points in space”
- Used to light dynamic/moving objects
- Drawbacks :
 - Can break batching
 - Applied per object, not per pixel → Bad for larger objects or more complex lighting
 - Placing light probes is cumbersome (tools can improve it)
- Alternatives :
 - Environment lighting
 - Light volumes



Light probes drawback : Applied per object, not per pixel

Example below : Tiny light source in the middle of the scene (at feet level)



Preparing the scene : Light volumes

Alternative to light probes: **Light volumes**.

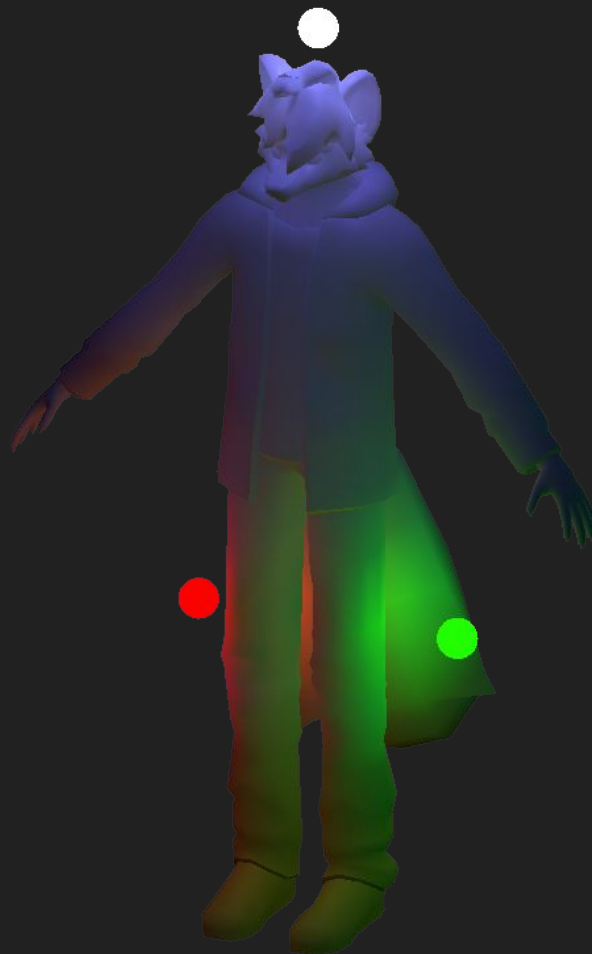
Offers per-voxel lighting → good for more complex lighting.

Generates a 3D texture to store the data.

Drawbacks:

- Larger build size
- Requires compatible shaders

“Overkill” for simple lighting setups

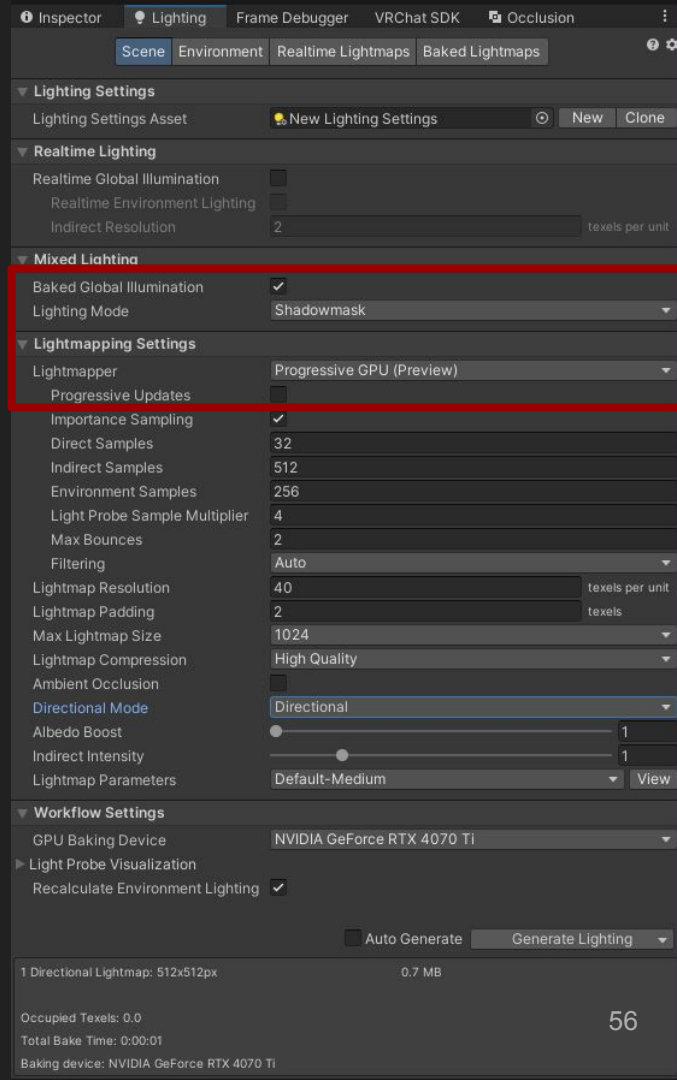


Light baking

Step 3 : Lightmapping settings

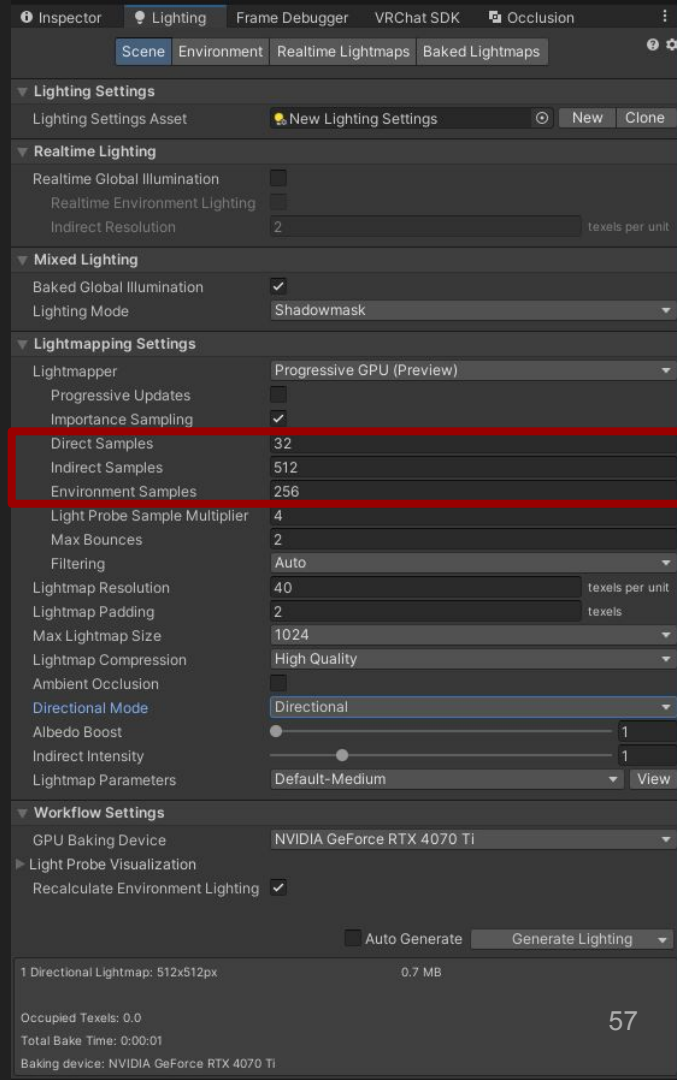
Step 3 : Lightmapping settings

- Lighting Settings Asset → New
- Lighting Mode → Shadowmask
 - Can bake indirect and direct lights
 - Shadow blending between RT and baked shadows
- Lightmapper : “Progressive GPU” is much faster.
- Progressive Updates → OFF (Faster baking time)



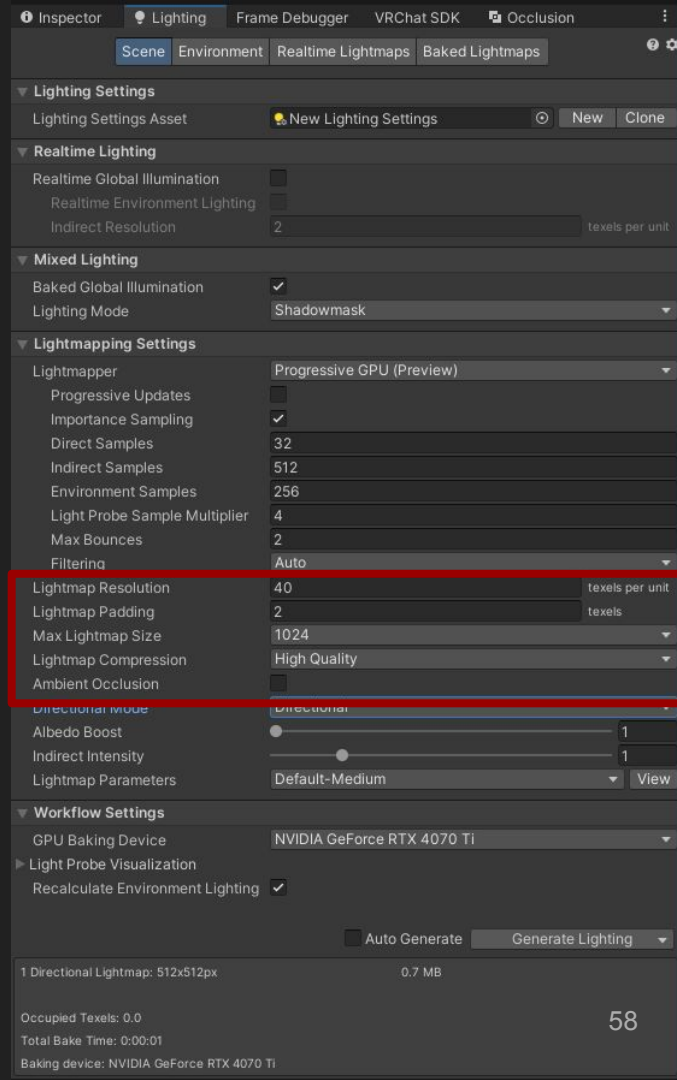
Step 3 : Lightmapping settings

- Samples : higher value = better quality, but increased baking time
 - Direct : Light sources (point/spotlight...)
 - Only really useful when working with soft shadows
 - Indirect : Bounced light, emissive materials
 - Should be set to a higher value
 - Environment : Light defined in the environment tab (skybox...)
 - Should be set to a higher value



Step 3 : Lightmapping settings

- Lightmap resolution (Texel density)
- Max Lightmap size : Higher resolution per lightmap requires more VRAM during baking
- Ambient occlusion to make corners/crevices darker



Ambient occlusion

Max distance (AO width)
0.5 2

Indirect
contribution
(AO intensity)

1

4



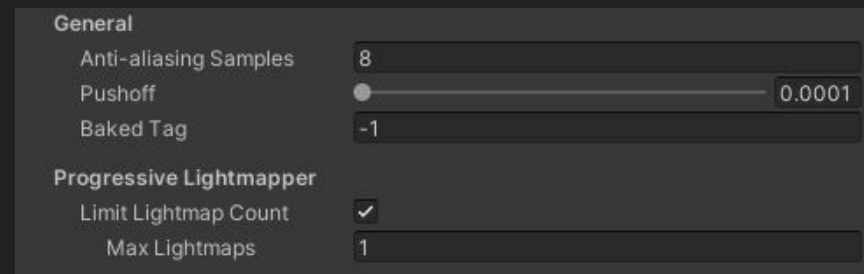
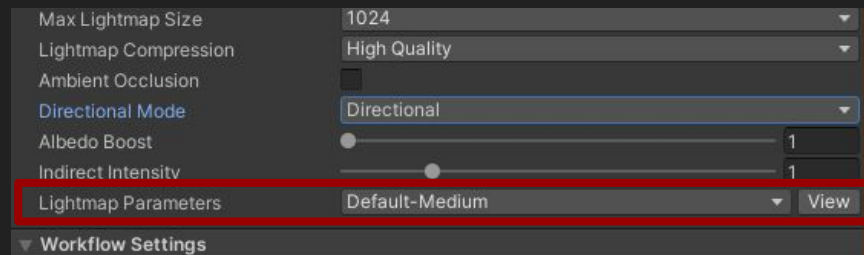
No AO



Step 3 : Lightmapping settings

Lightmap parameters:

- Limit Lightmap count :
 - ON : Reduces texel density if needed
 - OFF : No reduction in texel density



Lightmap settings

Directional mode : Non-directional

- No directional lightmaps are baked
- One HDR lightmap
- Normal mapped surfaces look flat
- Use this option when you don't use normal maps

Example : Brick texture looks flat even though it has a normal map



Lightmap settings

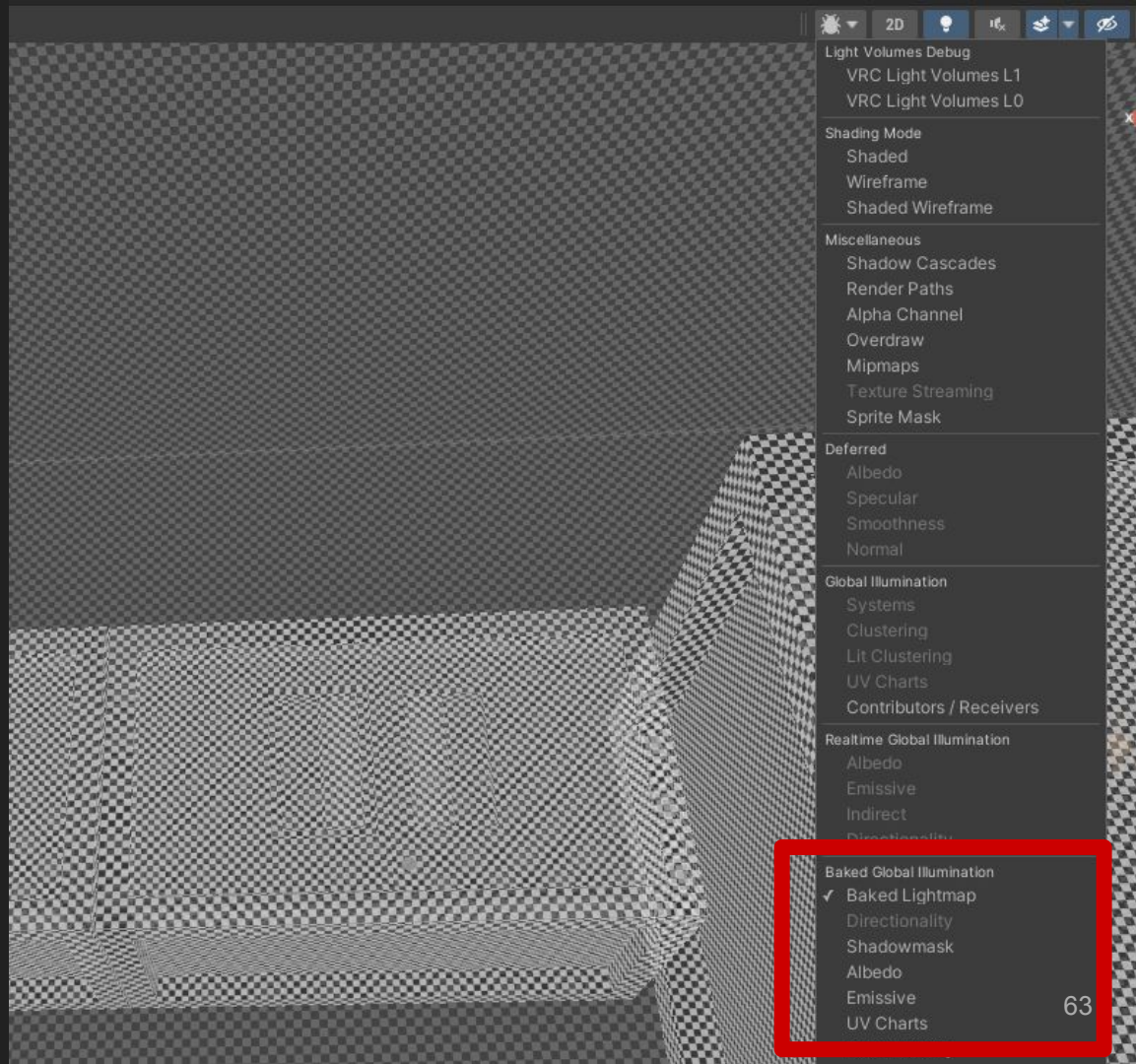
Directional mode : Directional

- One HDR lightmap + One directional map
 - Heavier option
- Great for Normal mapped surfaces
- Used by some shader features (baked specular...)



Lightmap settings

Preview Texel density



Light baking
Step 4 : Bake!

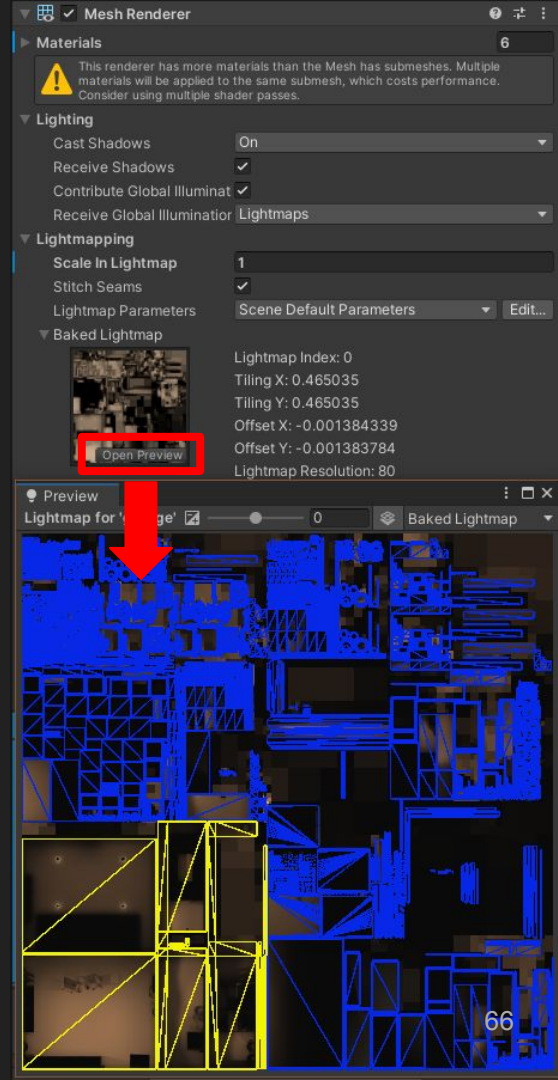
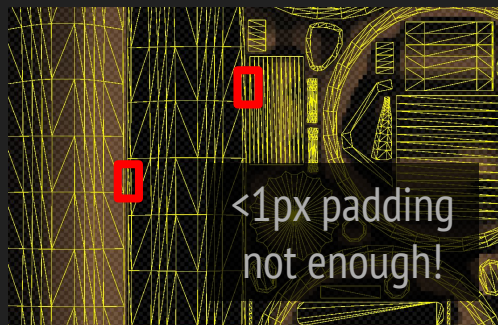


← Persp

After light baking

Use the lightmap preview window to check for issues:

- Resolution OK?
- Padding OK? No bleeding?
- Packing OK? No wasted space?
- ...



Lightmap bleeding caused by mipmapping

Mipmapping lightmaps.

Pros :

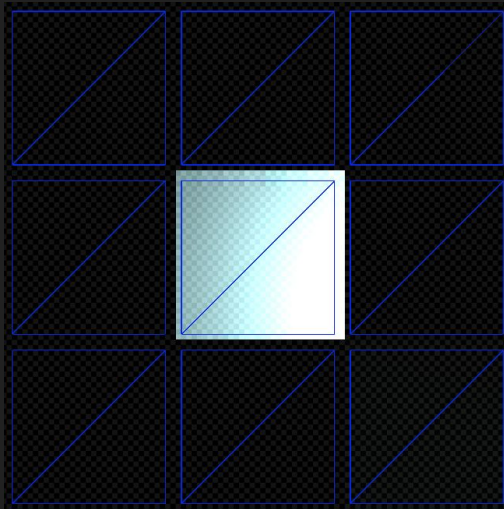
- Slightly better performance
- Less aliasing on higher poly meshes
 - Use LODs instead

Cons :

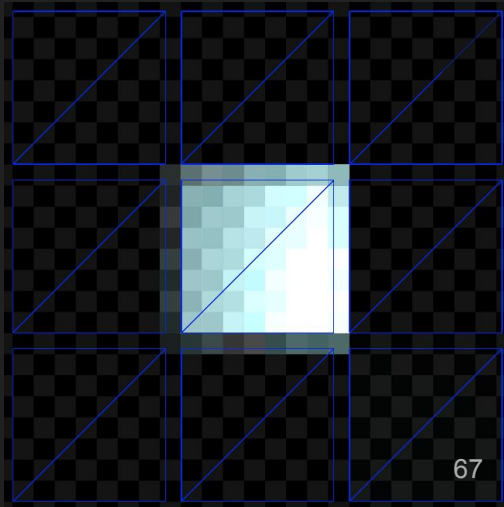
- Lightmap bleeding

Personal recommendation : Mipmaps OFF.

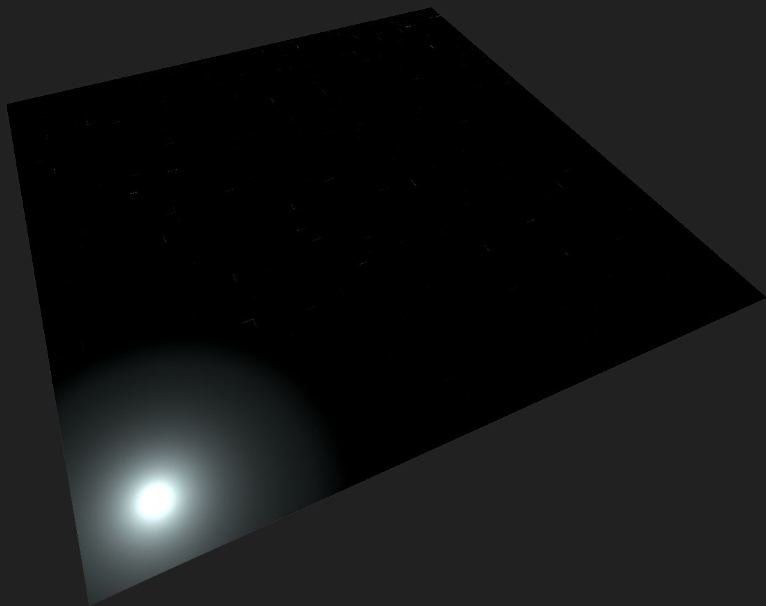
2048



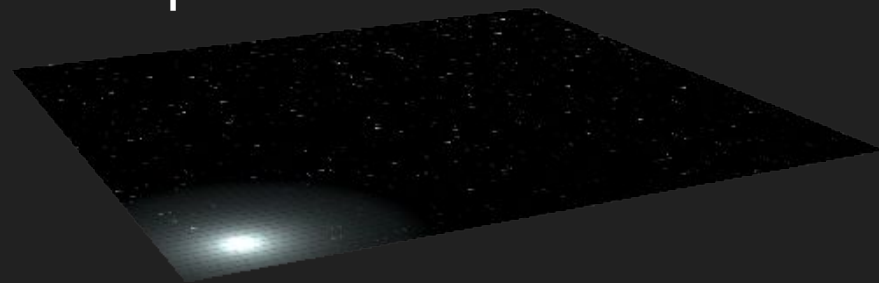
512



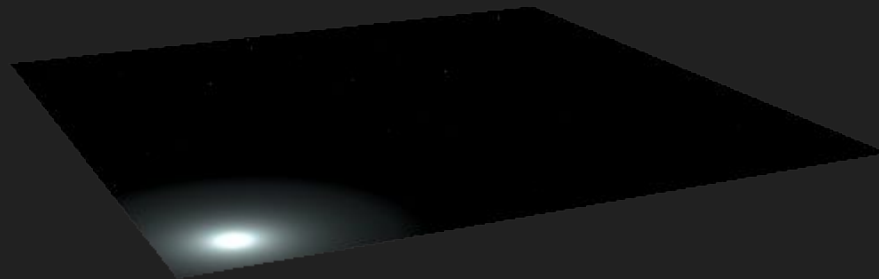
High-poly plane, each face on a separate UV island



Close distance



Far distance, mipmap ON

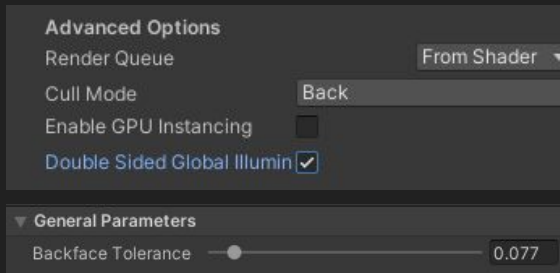


Far distance, mipmap OFF

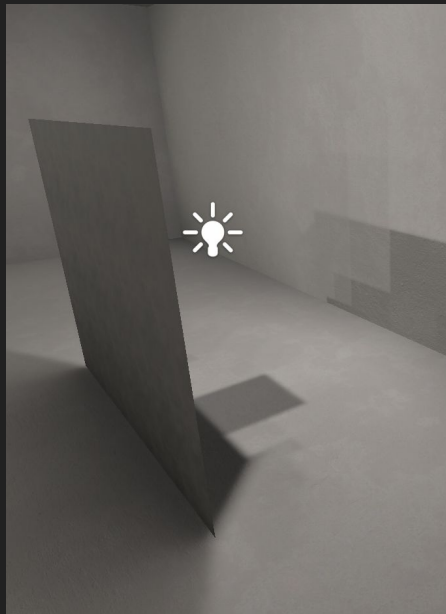
Common Baking Issues – Invalid Texels Artifacts

Lightmap artifacts can occur when rays hit back-facing polygons, causing Unity to mark texels as invalid.

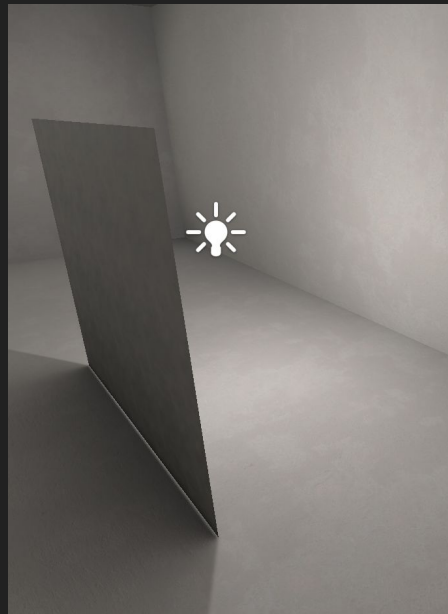
Fix: Enable “Double Sided Global Illumination” on the material of the object that has backfaces, or lower “Backface Tolerance” setting



Double sided OFF



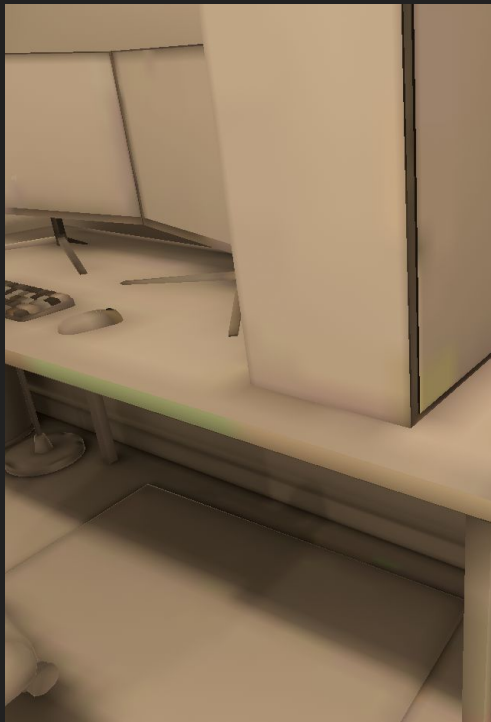
Double sided ON



Common Baking Issues – Colorful Texels

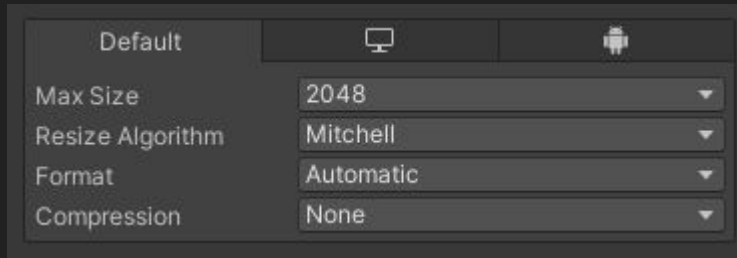
Usually caused by lightmap compression, mostly visible on white/black surfaces

Fix : Disabling compression (increases build size)

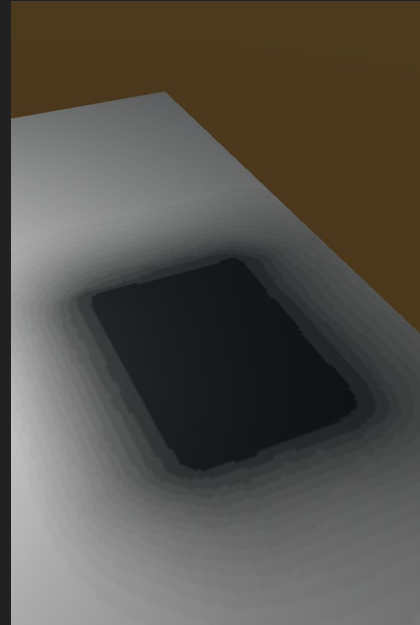


Common Baking Issues – Banding when baking a shadowmask

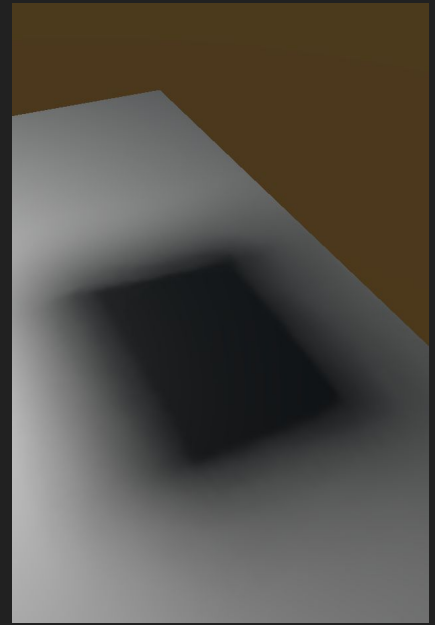
Fix : Select the shadowmask, and
disable texture compression
(increases build size)



Compression High



Compression Off



Common Baking Issues – No visible baked lights?

Some shaders don't support it!

Example : Poiyomi Toon

(Note : The “Poiyomi Toon World” variant supports baked light)

Poiyomi Toon (realistic shading)



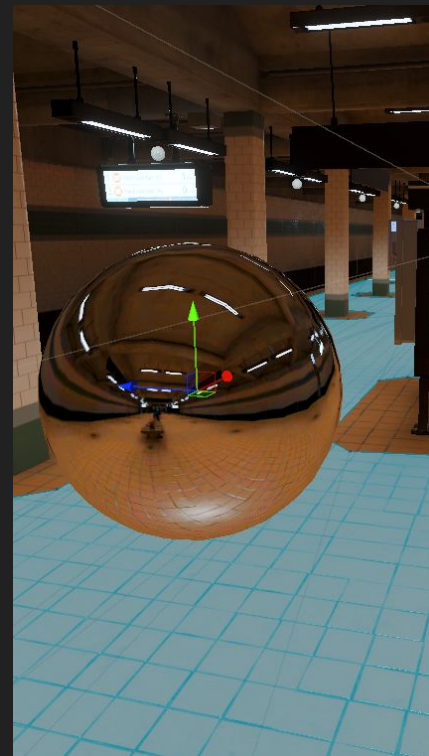
Unity Standard



Reflection Probe

Reflection probes

- Technique to add reflections in the scene
- Captures a spherical view of its surroundings in all directions
- Stored as a cubemap (6 faces)
- Used by objects to render reflections



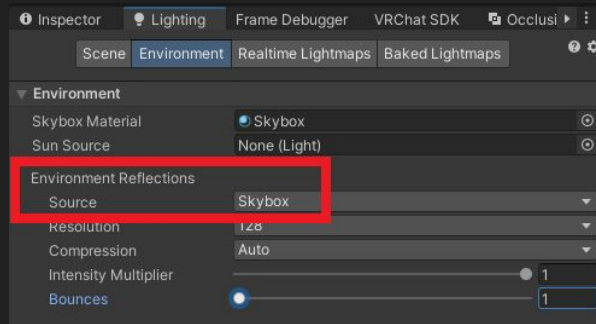
Reflection Probe

Drawbacks :

- Approximation
- Static environment
- Mostly look the best at certain positions
- Affects the whole mesh

Reflection Probe

Without reflection probes, reflections come from the skybox or the Environment Reflection fallback color.



No RP



With RP

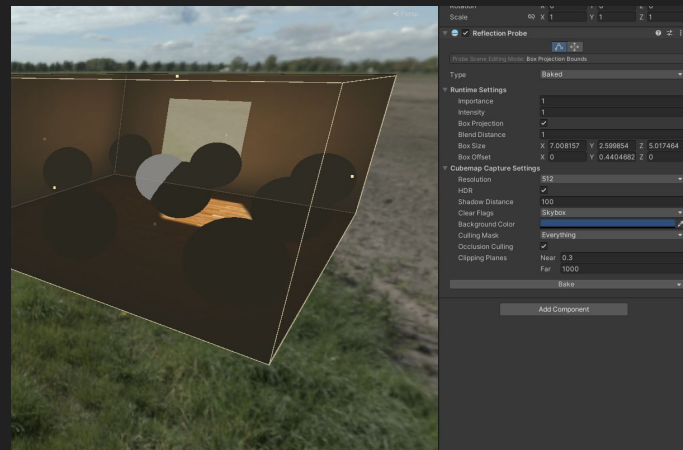
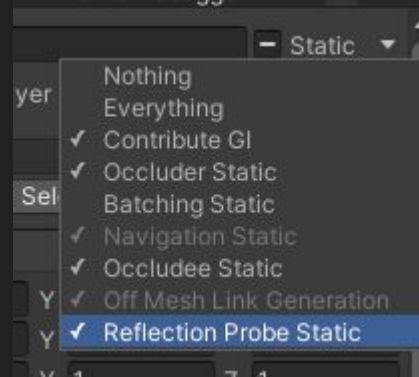


Reflection Probe (RP)

Setup:

- Put RP in the scene (eye height is good)
- Mark objects as “Reflection Probe Static” → included in baked reflections
- Define RP bounds → any object inside uses this RP

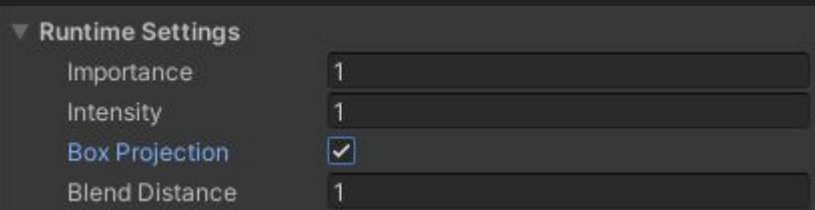
Click on “Bake”!



Reflection Probe

Box projection:

- Good for Indoor environments
- Prevents reflections from “sliding”
- Looks better in VR (proper parallax)
- But it's often not great in those cases:
 - Outdoor scenes (sky / distant environment)
 - Large open spaces (box projection has a finite distance)



Box projection ON



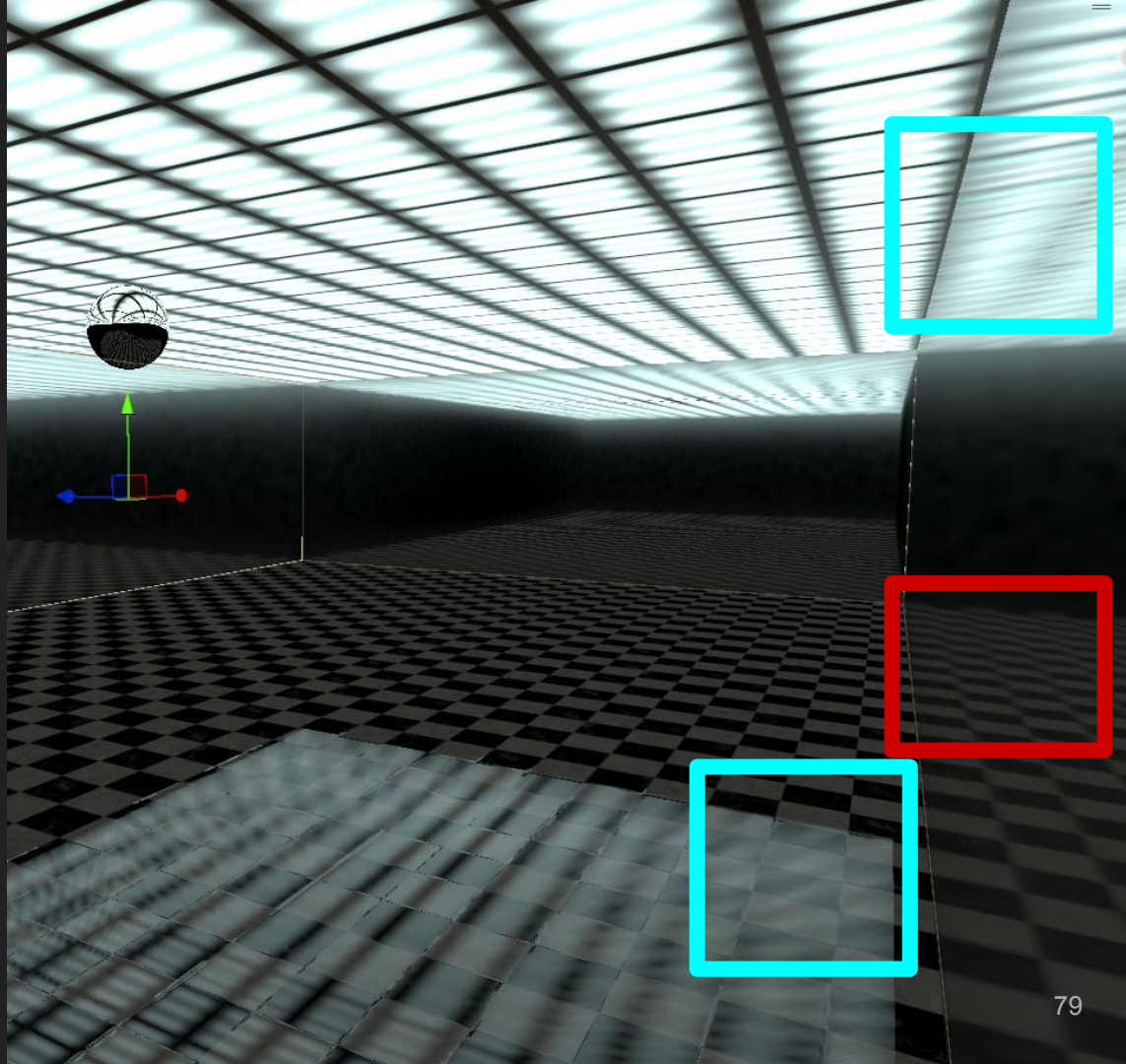
Box Projection OFF



Reflection Probe

When RP closer to the ceiling :

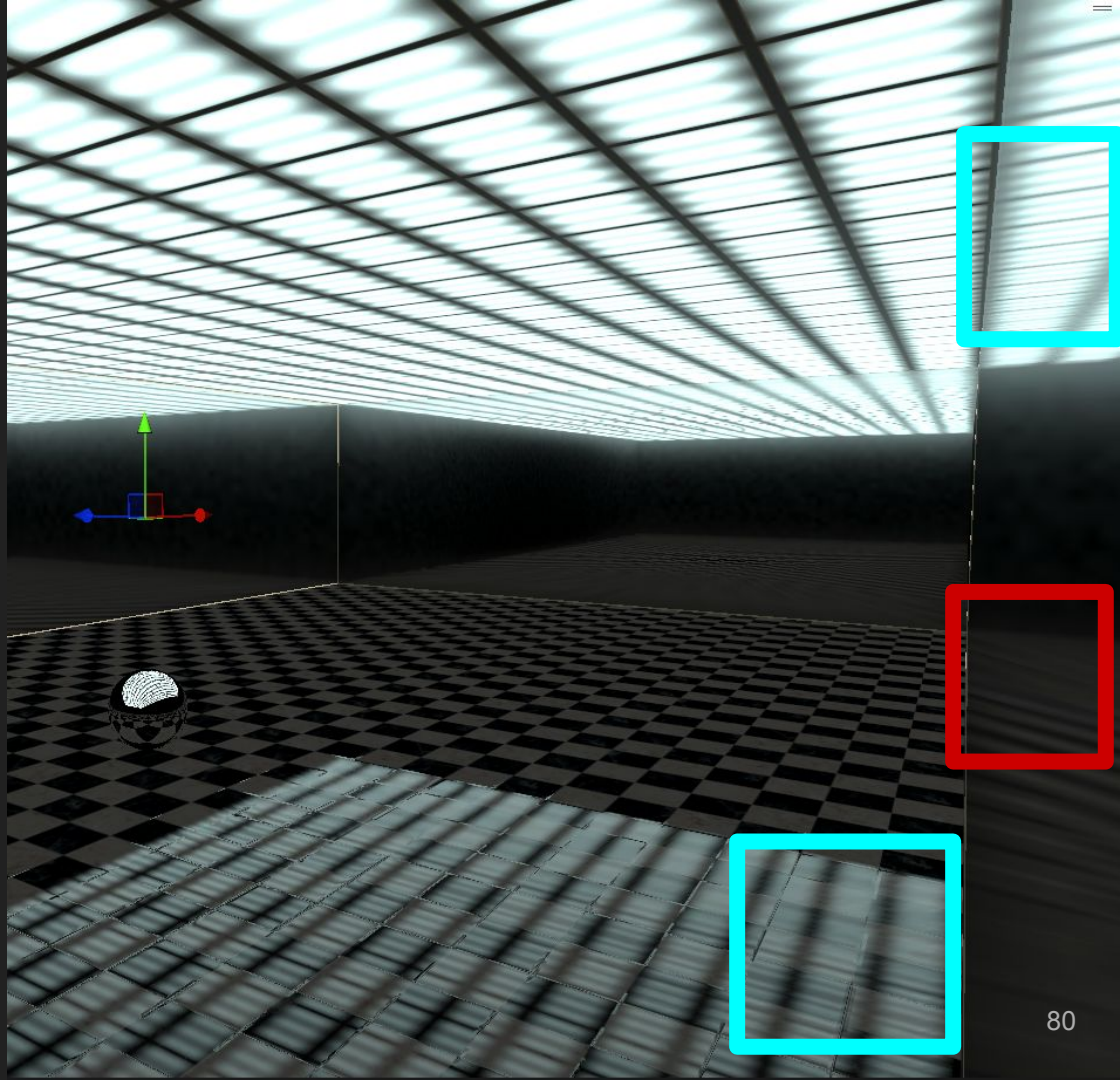
- RP Captures more from the floor → Red square has a higher resolution
- RP Captures less from the roof → Cyan square has a lower resolution



Reflection Probe

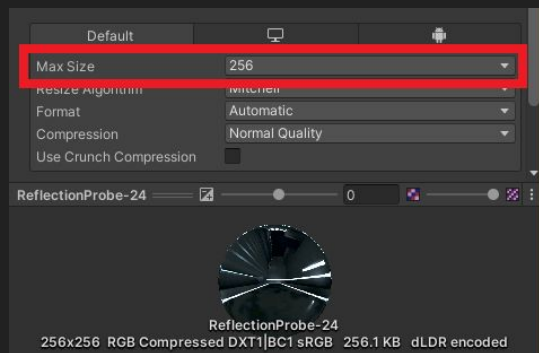
When RP closer to the floor :

- RP Captures less from the floor
→ Red square has a lower resolution
- RP Captures more from the roof
→ Cyan square has a higher resolution



Reflection Probe

Reflection probe quality can be improved by baking it at a higher resolution, then downscaling it in the import settings



Resolution 256



Resolution 1024 → 256



Reflection Probe - Box projection on Mobile

Box Projection is disabled on mobile due to VRChat's project settings, but some shaders still support Box Projection on mobile.

Example : Orel's Standard shader

Mobile Tweaks

Force Box Projection



Apply Mobile Color Correction



[Show Note](#)

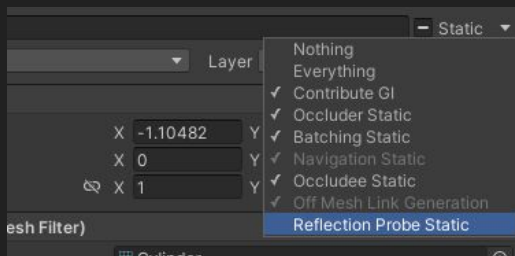
Screenshot taken on a mobile device
note the box-projected reflection.



Reflection Probe

If needed, an object can be excluded from a cubemap by unchecking “Reflection Probe Static”

Example : This cylinder, notice the reflection disappearing



Pillar included



Pillar excluded



Specular highlight disappears after light bake

Specular highlight from RT light

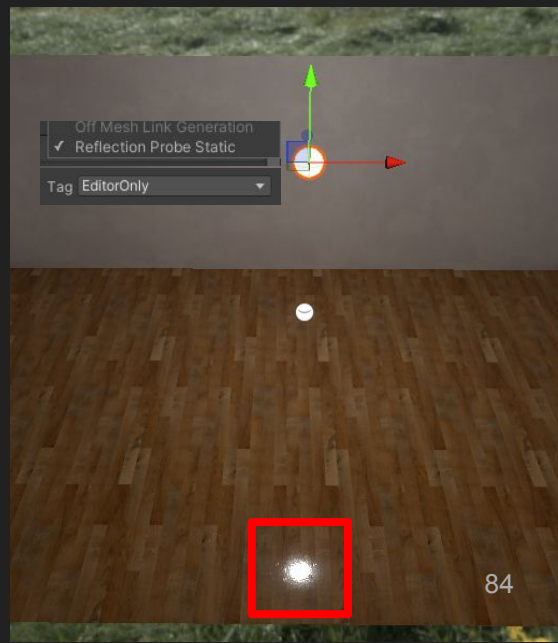


Baked : No more Specular highlight



Baked light

+ Added bright “proxy” sphere (EditorOnly, RP static)



An alternative to the Unity baker : Bakery

Bakery, what is it?

GPU lightmapper alternative:

- More features, more flexible...
- Per-light source sample count
- Used by many creators

But :

- Paid asset
- Doesn't work on Linux
- Only for NVidia GPUS

Bakery - GPU Lightmapper by Mr F

Settings mode Last bake: 00h 45m 31s

Experimental

Render mode

Full Lighting

Directional mode

Dominant Direction

Light probe mode

L1

Atlas packer

Default

Asset UV processing

Adjust UV padding

Unwrapper

Default

Denoiser

Optix 7

▼ Lightmapping tasks

- ☒ Export geometry and maps
- ☒ Adjust sample positions
- ☒ Unload scenes before render
- ☒ Update unmodified lights
- ☒ Update modified lights and GI
- ☒ Denoise
- ☒ Denoise: fix bright edges
- ☒ Fix UV seams

Auto-atlasing

- ☐ Split by baked tag
- ☐ Split by scene
- ☐ Remove duplicate lightmaps
- ☐ UV padding: increase only

Texels per unit: 5

Max resolution:

2048

Min resolution:

2048

► Scale per map type

▼ Checker preview

☐ Show checker

Global Illumination

Bounces: 5

Samples: 64

Hacks

Emissive boost: 1

Indirect boost: 1

Backface GI: 1

▼ Ambient occlusion

Intensity: 1

Radius: 0.75

Samples: 32

Softer blend: ☐

☐ Dither transparency

▼ Performance

☒ RTX mode

☐ Export terrain trees

☐ Terrain optimization

Light Mesh batch limit

0

☐ Compress volumes

Samples multiplier

1/1

Tile size

512

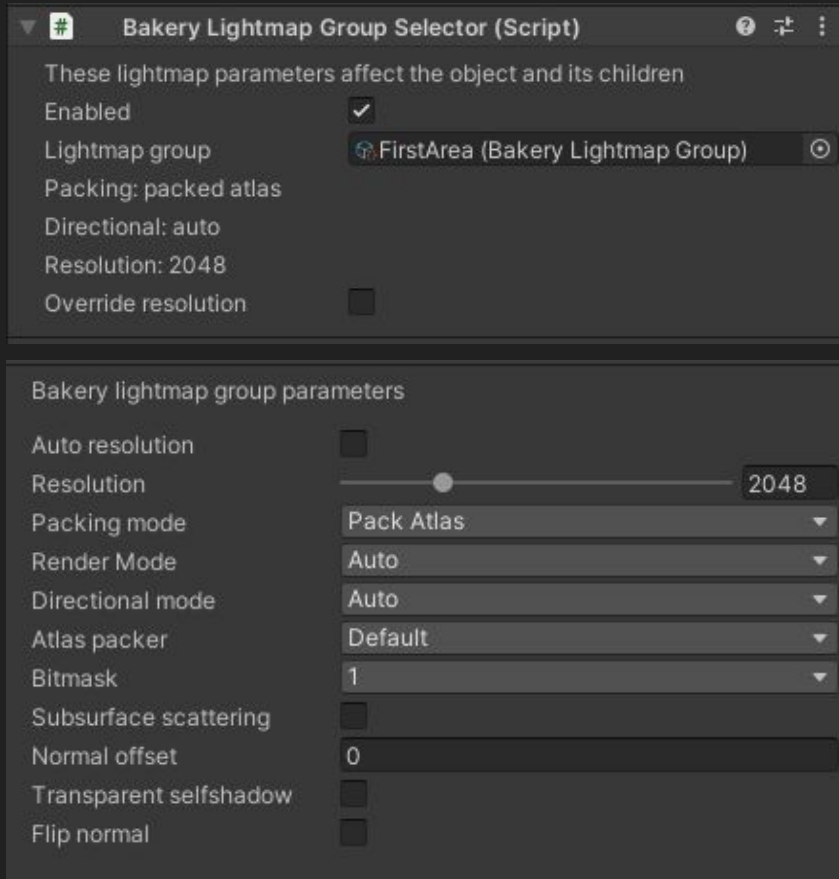
86

Bakery lightmap group selector

- Allows overriding Bakery-related settings in specific areas
- Ensures the object and its children are baked into the same lightmap

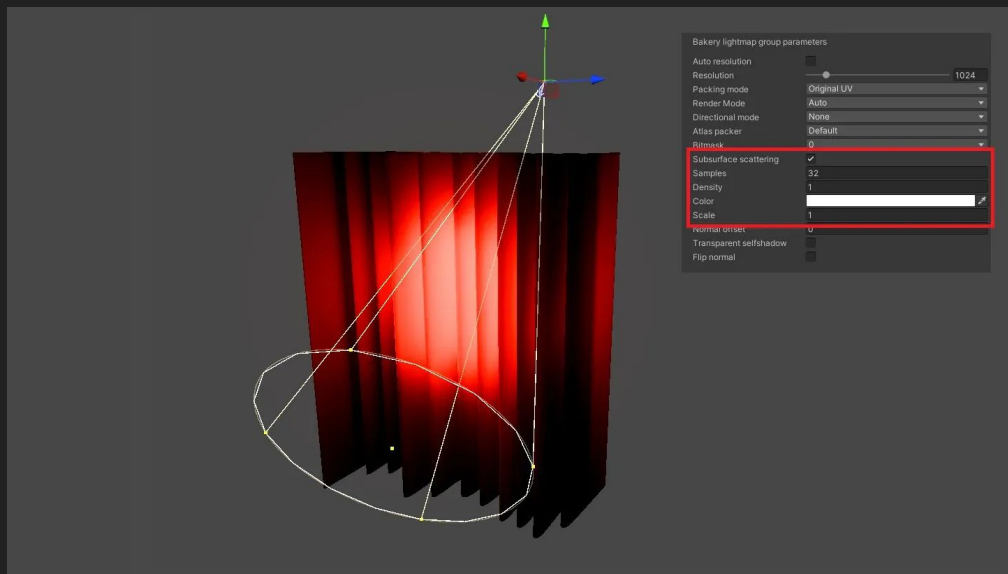
Additional features :

- Bitmask setting
- Subsurface scattering (Simulates light scattering through a mesh) for glass, foliage...



Subsurface scattering

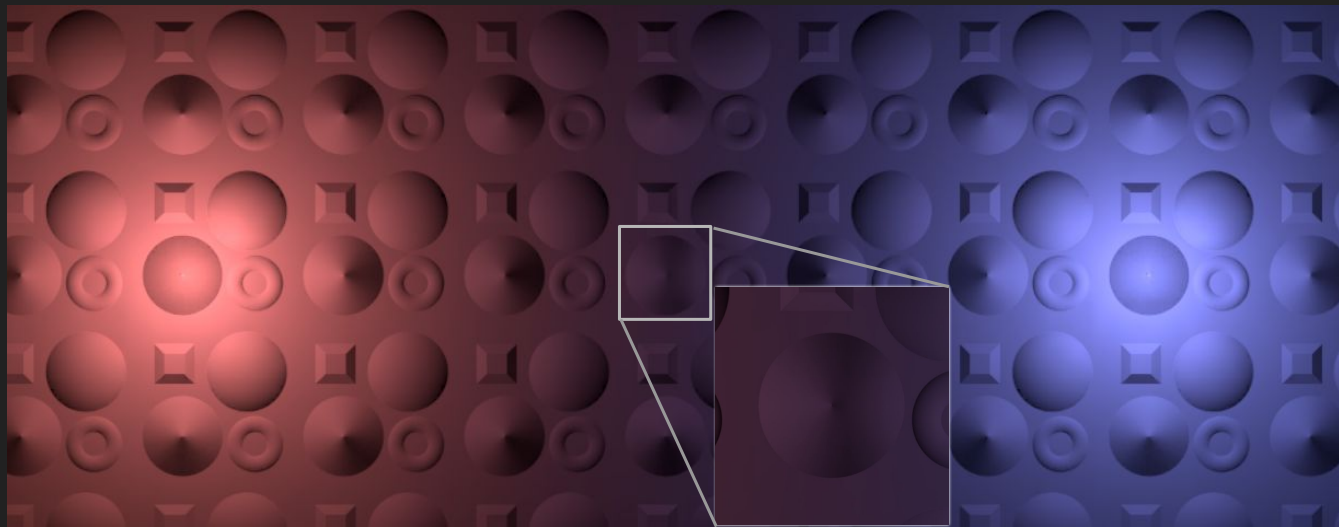
Simulates light scattering through a mesh



Bakery Directional mode

Dominant Direction: Also used by the Unity baker, only support a single dominant light direction. Bakes one HDR lightmap + one dom. dir. lightmap (RGBA).

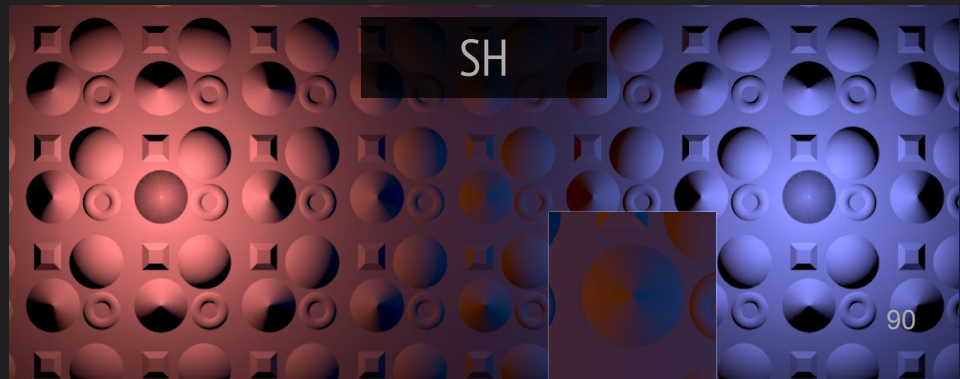
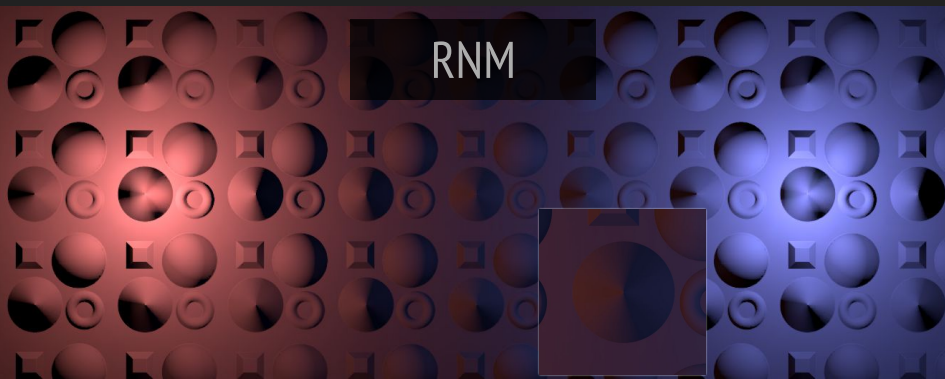
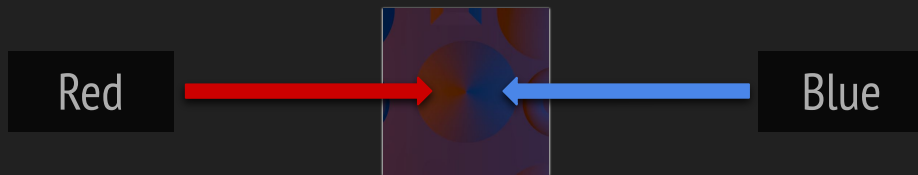
Can look flatter than the alternative, but *can* be compensated by increasing the normal strength on the material.



Bakery Directional mode

SH/RNM: Support multiple light directions and colored lighting information (notice below the red and blue colors not mixing between both light sources), but bakes much more lightmaps.

Requires custom shaders and custom scripts (<https://github.com/lvaj1/Udon-Bakery-Adapter>), and often overkill.

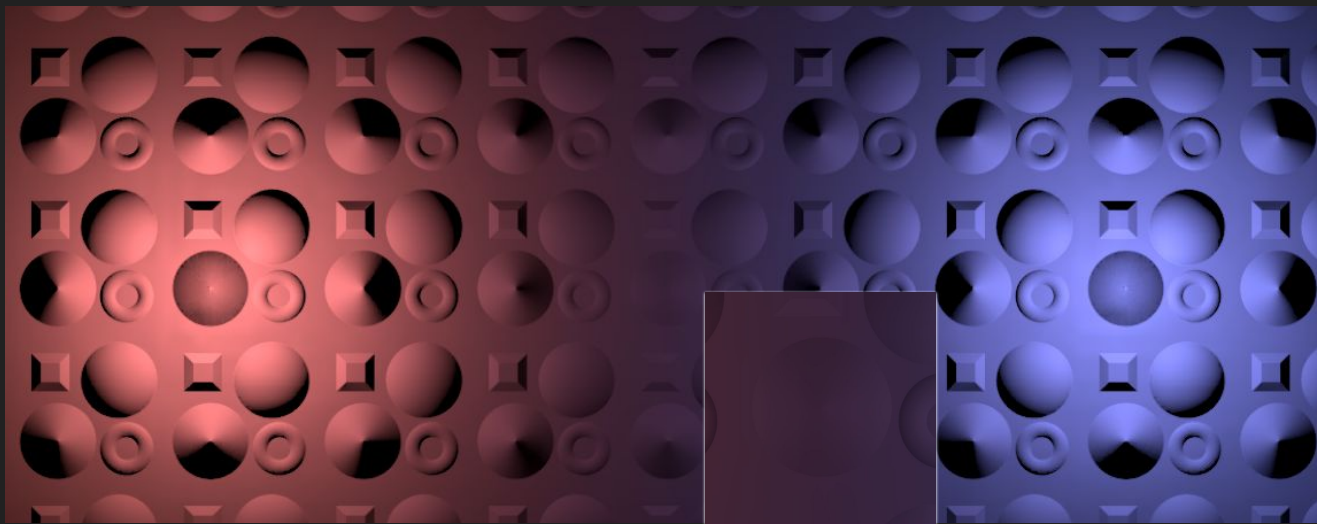


Bakery Directional mode

MonoSH: Alternative to dominant direction. Bakes one HDR lightmap + one directional lightmap (RGB texture containing SH coefficients).

Can look better or worse : Something to try on a case-by-case basis.

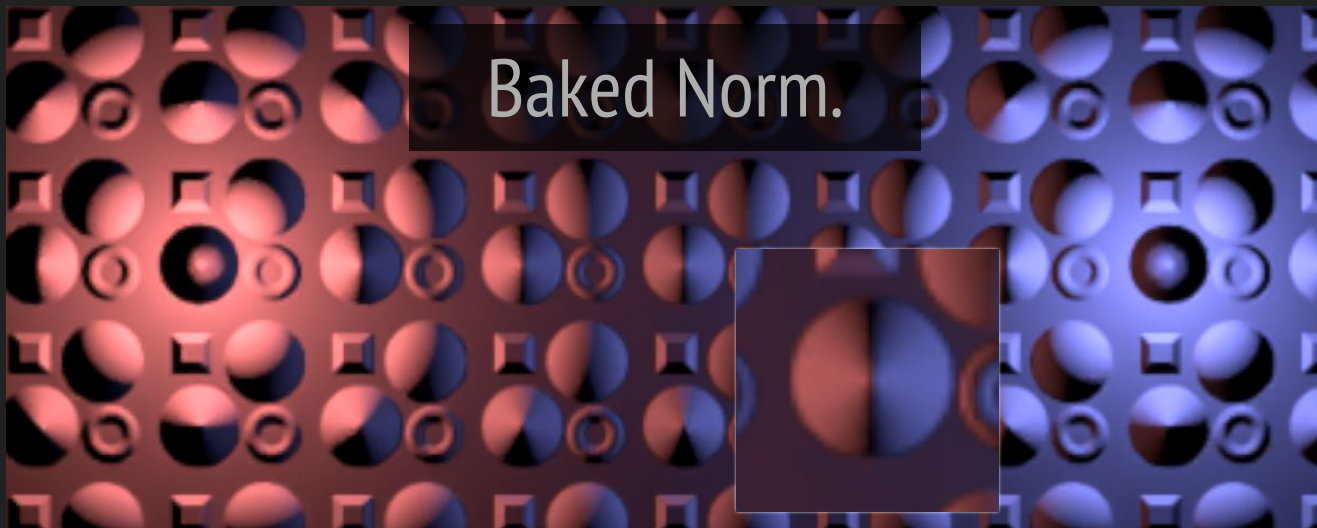
Requires custom shaders to work, but most shaders created by the VRC community supports it.



Bakery Directional mode

Baked Normal Map: Shadows cast by the normal map baked directly into a lightmap. Support multiple light directions and colored lighting information

Unusable in most VRChat worlds, as it only looks good when baking high-res lightmaps. Good for rendering.



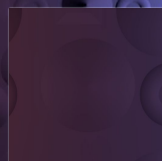
Dom. Dir.



RNM



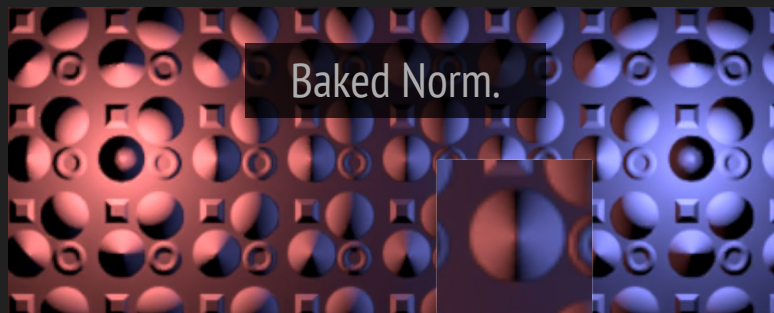
MonoSH



SH



Baked Norm.



Supports multiple directions



Requires custom shader



Requires custom scripts
(<https://github.com/lvaj1/Undon-Bakery-Adapter>)



Bakery - “Physical Falloff” toggle

More physical accurate light propagation: light is brighter at the start, and gets darker quicker

“Physical Falloff” ON, “falloff min size” 0.1



“Physical Falloff” OFF

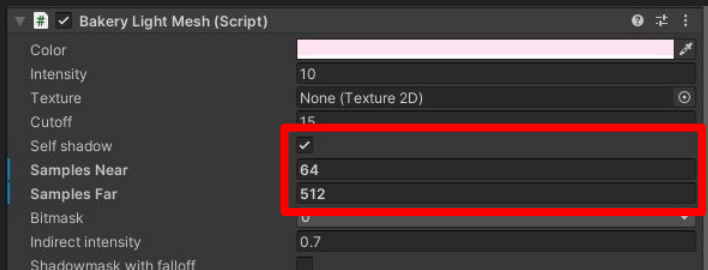


Light baking

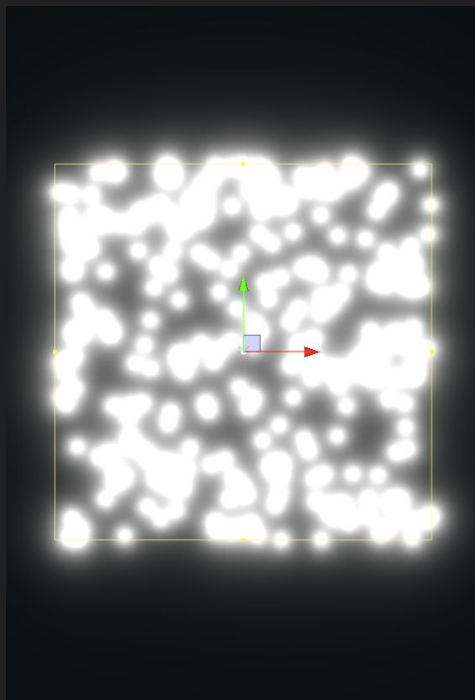
Common issues

Noisy emissive lights? Noisy area lights?

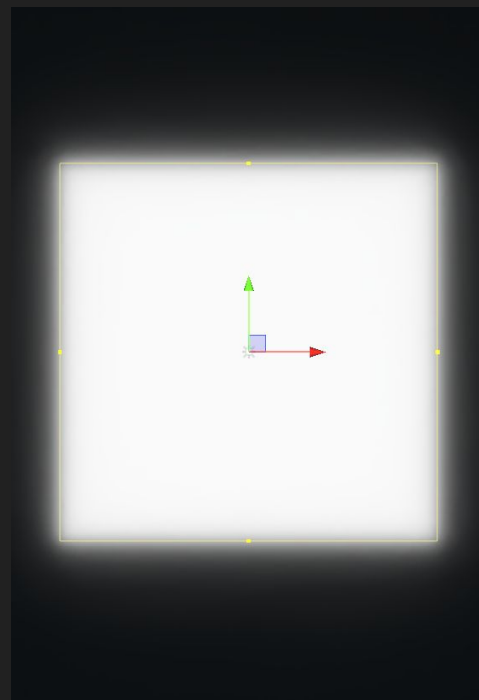
- Use the Bakery light mesh component instead
 - Put emissive material on a separate mesh
 - Increase sample count setting
 - Set “Self shadow” ON



Area light, self shadow OFF

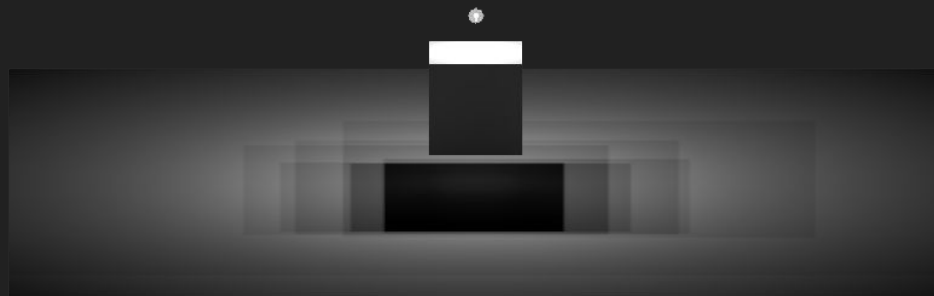
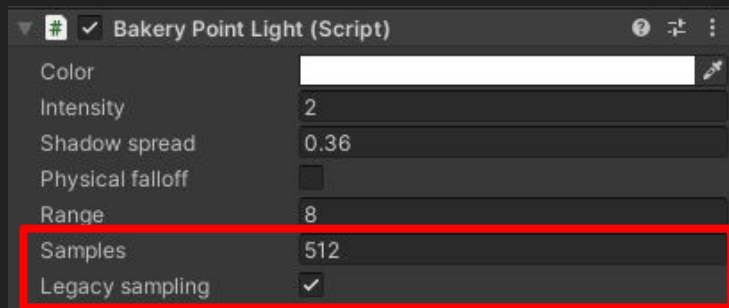


Area light, self shadow ON

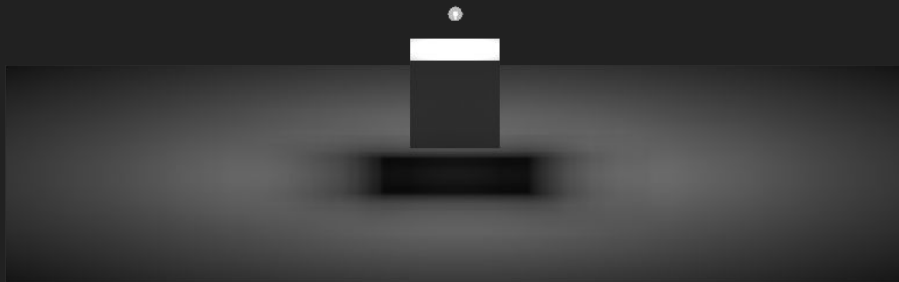


Bakery - Low quality soft shadows

Increase Sample size on the light source component, or disable “Legacy Sampling”



8 Samples (Default)



512 Samples

Light baking on avatars

Light baking with avatars - Bakery

For baking emissive masks, AO...

Bakery highly recommended!

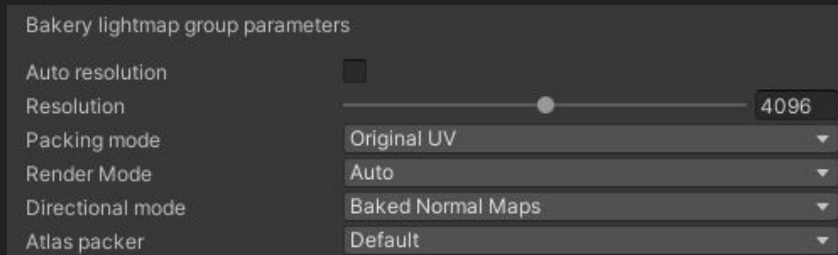
Blender export (as separate mesh recommended):

- All materials on a separate mesh
- Delete the second UV channel
- No overlapping islands on the original UV!
- “Generate lightmap UV” OFF

Unity scene : No light sources, no environment lighting

Bakery Lightmap group component on each Mesh

Renderer



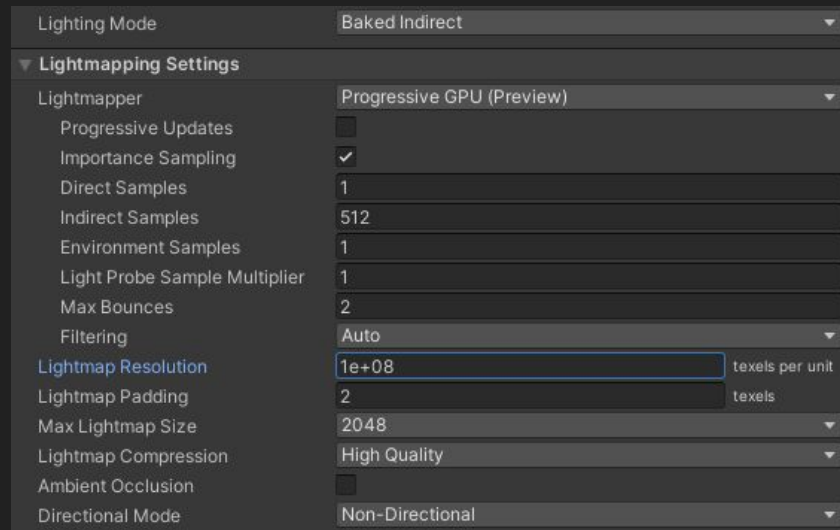
Light baking with avatars - Unity baker

Unity baker doesn't support baking on original UV (Unity still tries to pack islands).

Unity lightmap settings

- Non-directional
- Very high lightmap resolution

Rescale lightmaps after bake (Gimp...) might be needed...

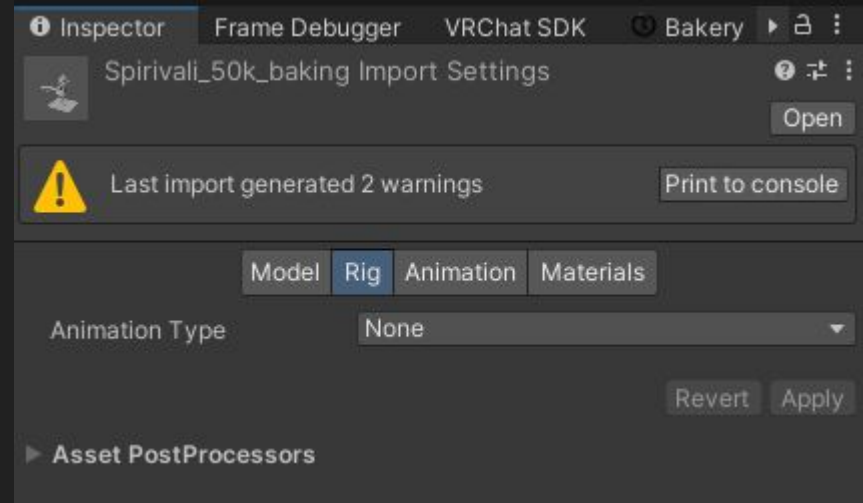


Light baking with avatars - Import settings

Only mesh renderers can be baked!

Skinned meshes can be converted to mesh renderers in the import settings.

Set “Animation type” to “None”.





Directional mode : None



Directional mode : Baked normal maps

Light baking with avatars

You can bake a texture mask by baking a lightmap using 3 light sources/emitters **R**, **G** and **B**





Shaders and shader features

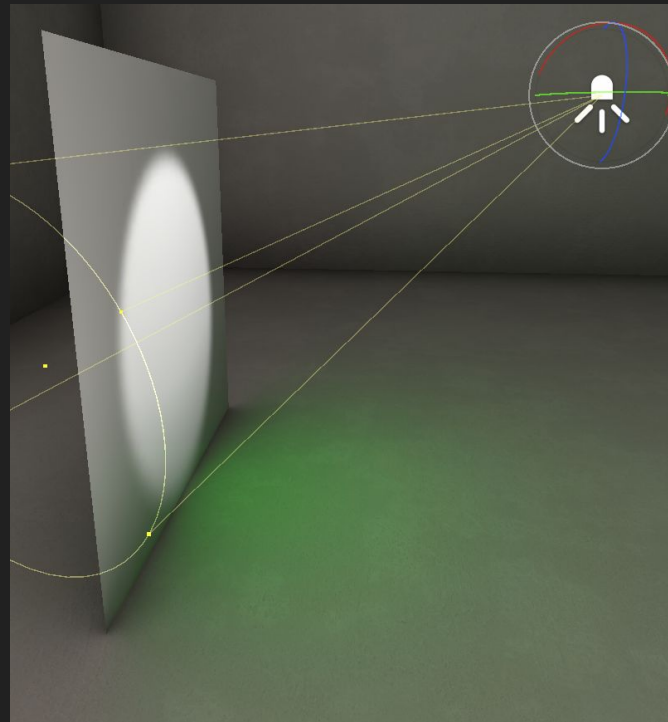
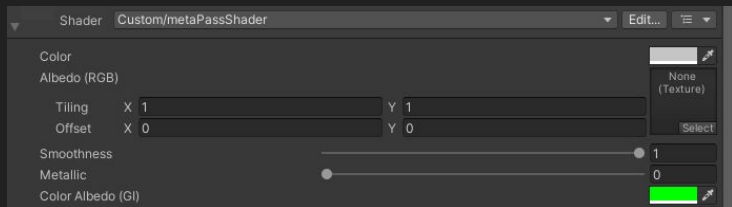
Meta Pass

Meta Pass

“Meta Pass” is a shader pass used by the lightmapper to read a material’s properties during baking. This allows materials to use custom values for baked lighting without affecting their visible appearance.

Example: This white wall reflects green indirect light because “Color Albedo (GI)” is set to green.

Alternative : EditorOnly wall



Meta Pass

Adding the property “_TransparencyLM” to a Shader allows us to add light transmission properties to a material

Example : Light shining through stained glass, projecting the image on the floor

Note : On Bakery, use the “Bakery Mesh filter” component instead.



Which PBR shader to choose?

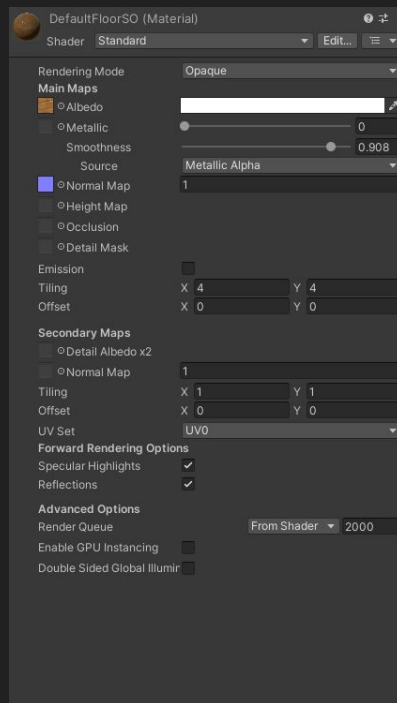
Silent Filamented - For its simplicity

- Inspector very similar to Unity Standard
- Uses the same color channels as Unity Standard
 - Red : Metallic, Green : AO, Smoothness : Alpha

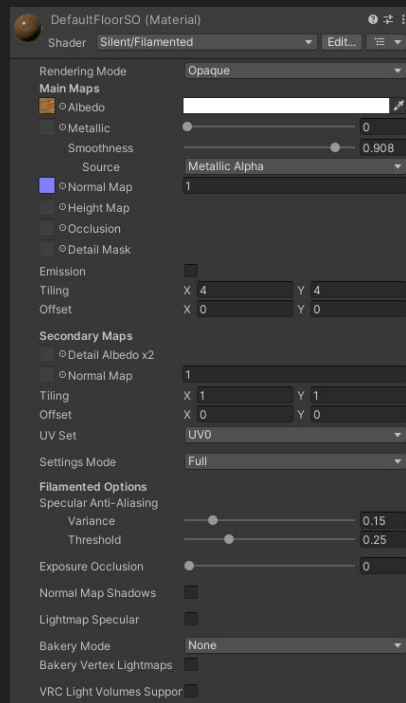
It looks really good

<https://gitlab.com/s-ilent/flamented>

Unity Standard



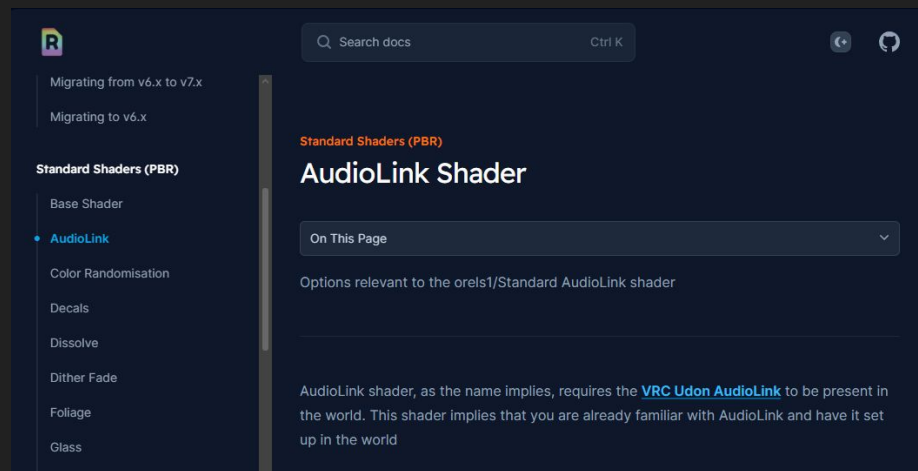
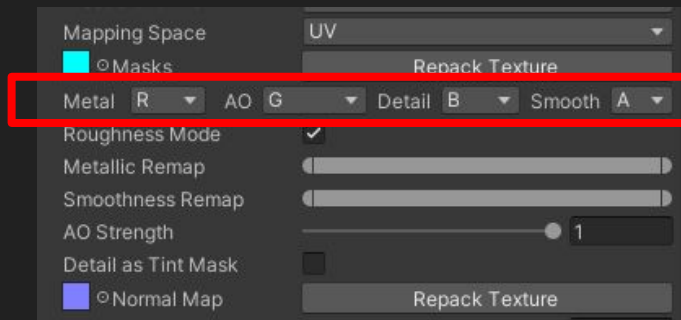
Silent Filamented



Orels1 - For the convenience

- Texture packer, color channel selector, metallic/smoothness remap...
- Amazing documentation
- Possibility to write your own orels shader modules (HLSL), and build your own shader.

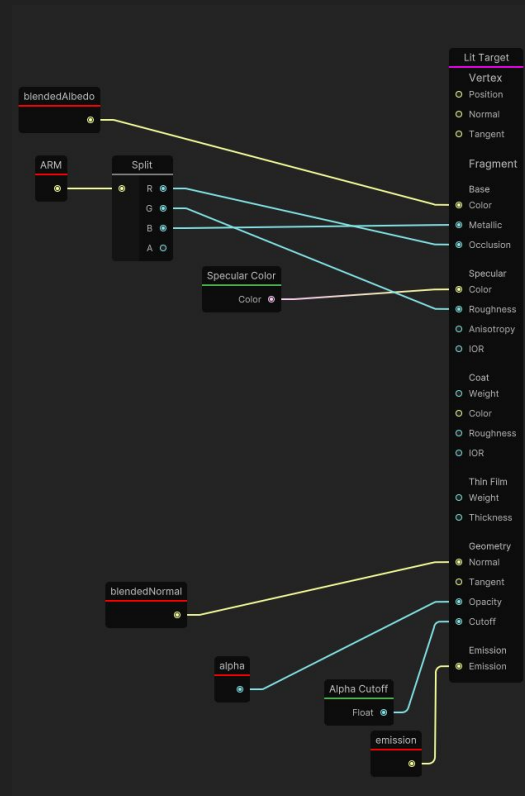
<https://shaders.orels.sh>



z3y (Graph)Lit - If you want to learn shader development

- Shaders made using Shader Graph, easy to modify, includes many nodes...
- Unique features like VRCTrace, Clustered BIRP

<https://github.com/z3y/Graphlit>



z3y (Graph)Lit - VRCTrace <https://github.com/z3y/VRCTrace>

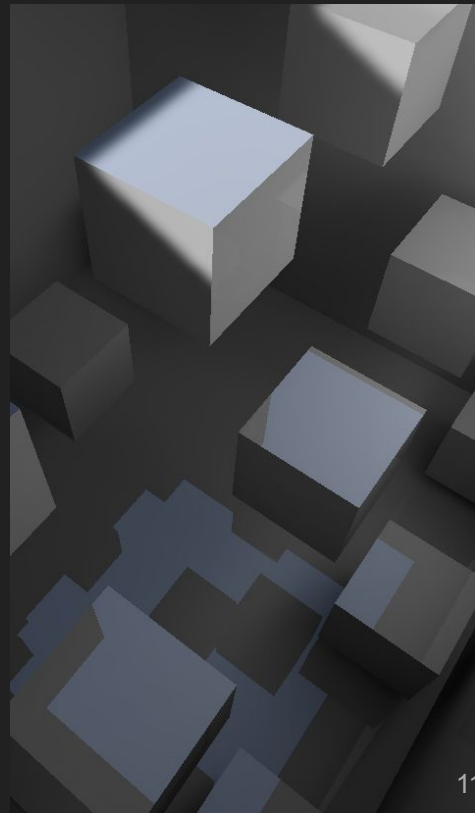
Software GPU Ray Tracing for VRChat.

Personal opinion : Works the best on smooth surfaces
(water, glass...), lot of aliasing on rougher surfaces.

Interesting alternative to screen-space reflection, although
it doesn't affect dynamic objects including avatars.

Demo world : 水鏡 Mizukagami RTX

https://vrchat.com/home/world/wrld_f7fa38b7-947a-4e34-a3e1-e082a4eb5b39/info



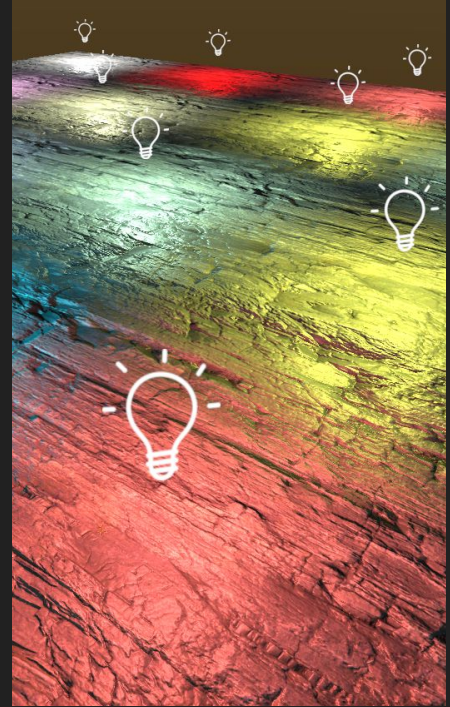
z3y (Graph)Lit - ClusteredBIRP <https://github.com/z3y/ClusteredBIRP>

Alternative to RT light sources.

System allowing you to add up to 128 fake RT light sources without flickering issues

Demo world

https://vrchat.com/home/world/wrld_cb7fd9cd-55d5-4e48-8157-46322cfaf61c/info



Mochie Standard - Most features out of the box

- Feature rich base shader (vertex manipulation, Subsurface scattering, rain, puddles, SSR...)
- Mixed sampling mode (example : “Primary texture” Triplanar, and “Details” Stochastic)

<https://github.com/MochiesCode/Mochies-Unity-Shaders>



Useful shader features

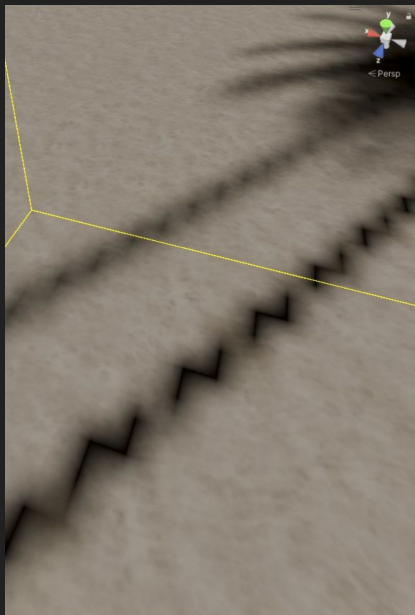
Shader features - Bicubic Lightmap Sampling

"Bicubic Lightmap Sampling" smooths lightmaps, allowing them to be baked at a lower resolution or texel density, which also reduces world size.

Example : Mochie shader.



Bicubic Sampling OFF



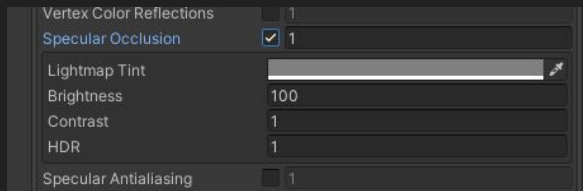
Bicubic Sampling ON



Shader features - Specular occlusion

“Specular Occlusion” reduces indirect lighting (Reflection probes, light probes etc.) in areas where indirect lighting should be partially blocked.

Example : Mochie shader.



Specular occlusion OFF



Specular occlusion ON

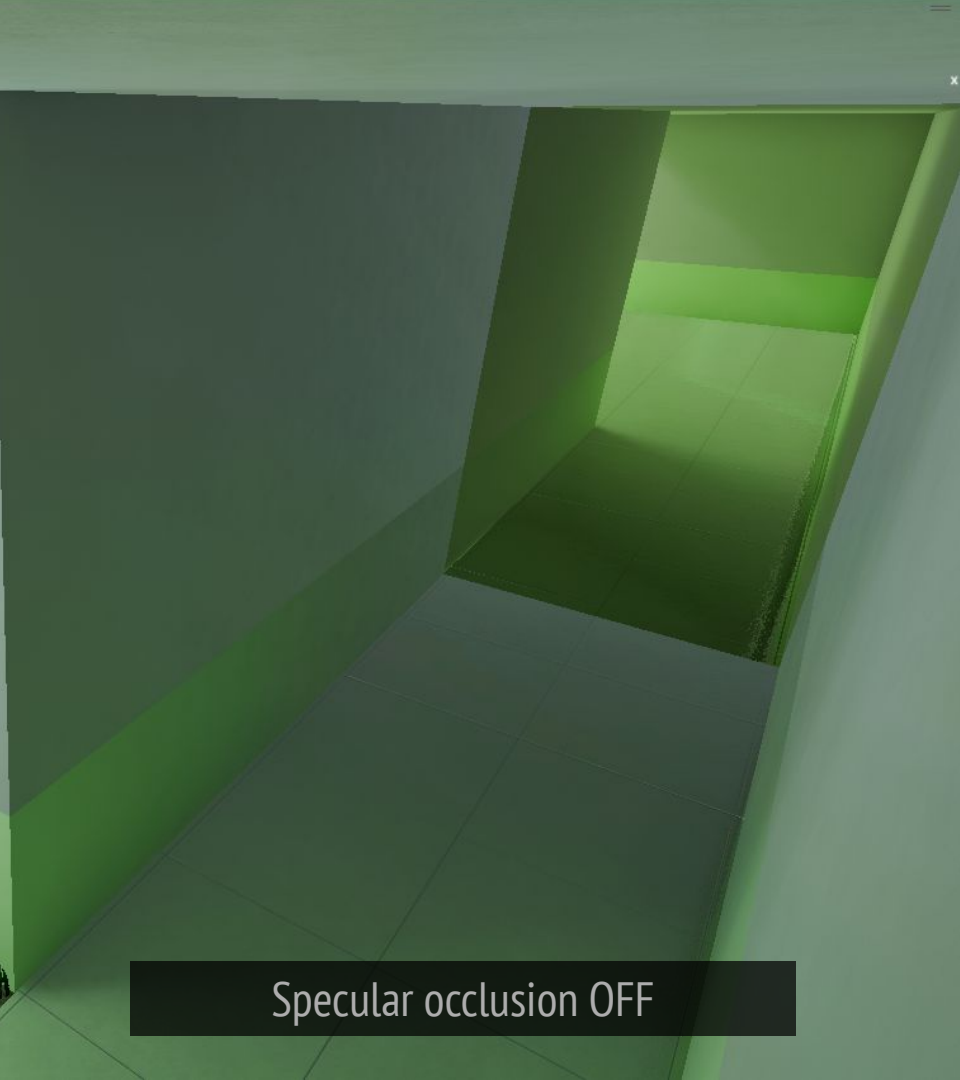




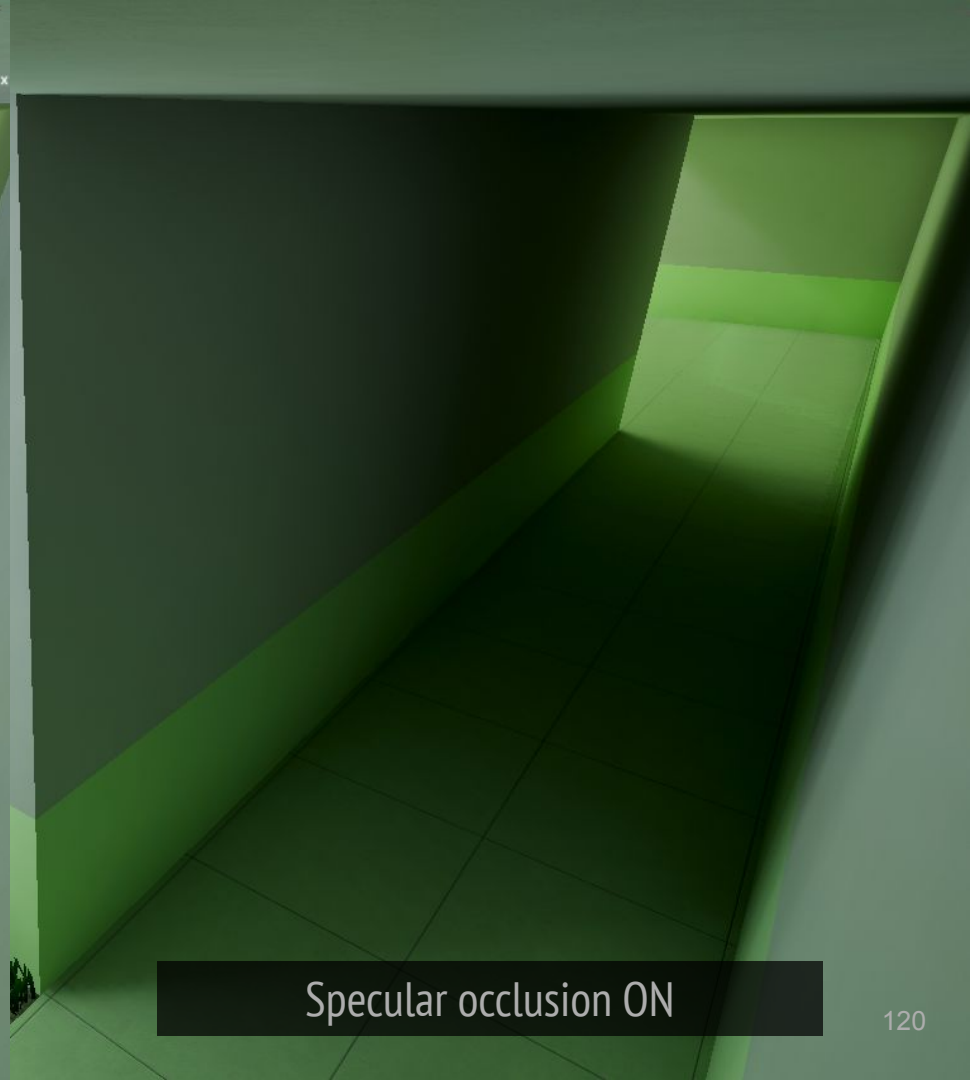
Specular occlusion OFF



Specular occlusion ON



Specular occlusion OFF



Specular occlusion ON

Caution : Specular occlusion sometimes removes reflections where we actually want reflections

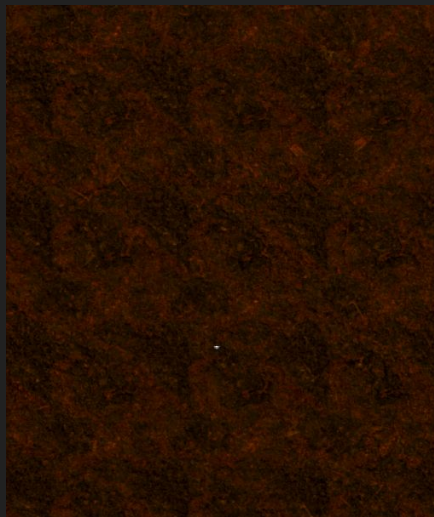


Shader features - Stochastic Sampling

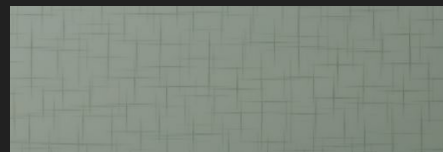
Stochastic sampling adds randomness to break visible tiling and repetition.

But : Fails on structured/repetitive patterns (bricks, tiles...) where randomness breaks alignment. Can remove some details.

Stochastic Sampling OFF



Stochastic Sampling ON



Shader features - Specular Antialiasing

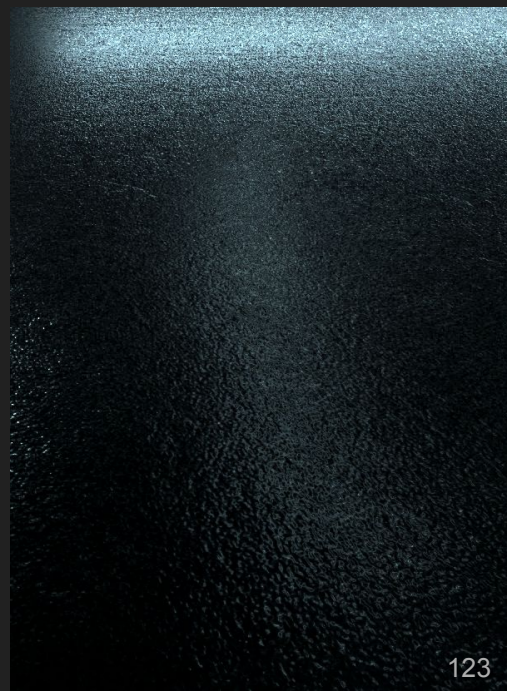
Reduces shimmering from specular highlights using techniques like GSAA, roughness filtering...

Example : Orels GSAA implementation

GSAA OFF



GSAA ON



Shader features - Lightmapped Specular

The shader uses the directional lightmap to guess where the lightsource is placed in the scene, and adds specular highlights on the surface

Can look weird when there are multiple light sources.

Baked Specular OFF



Baked Specular ON



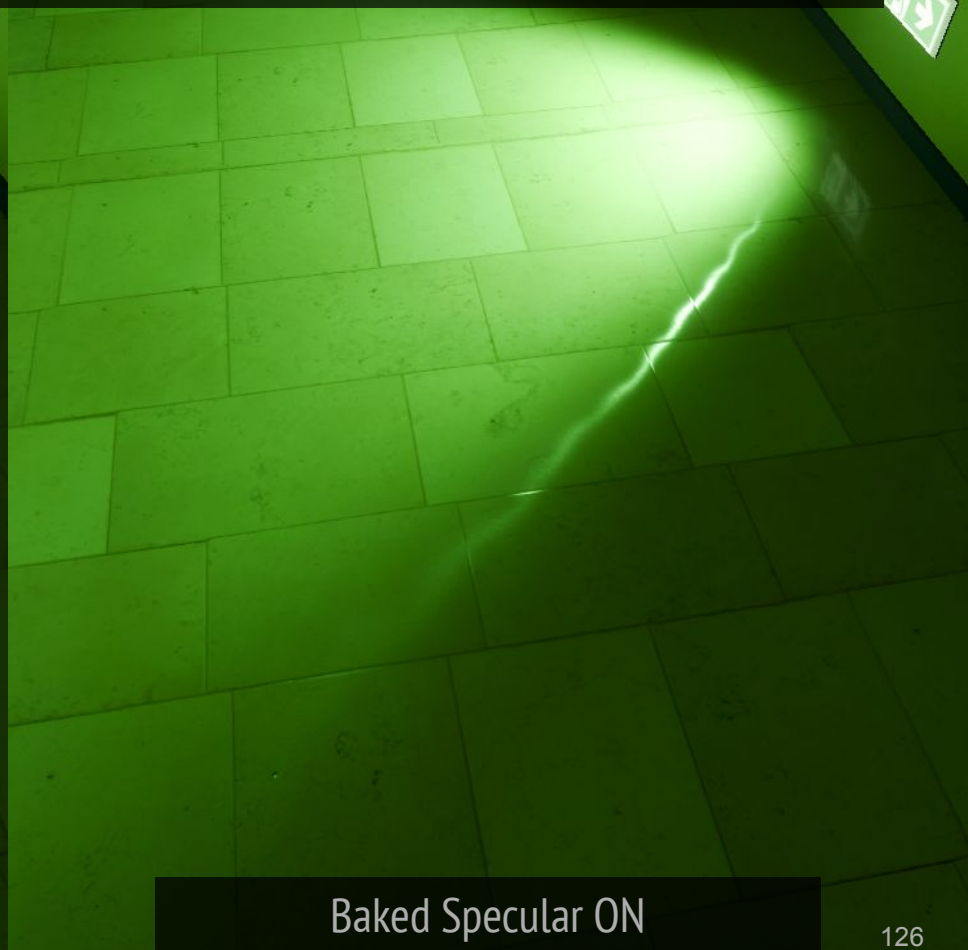
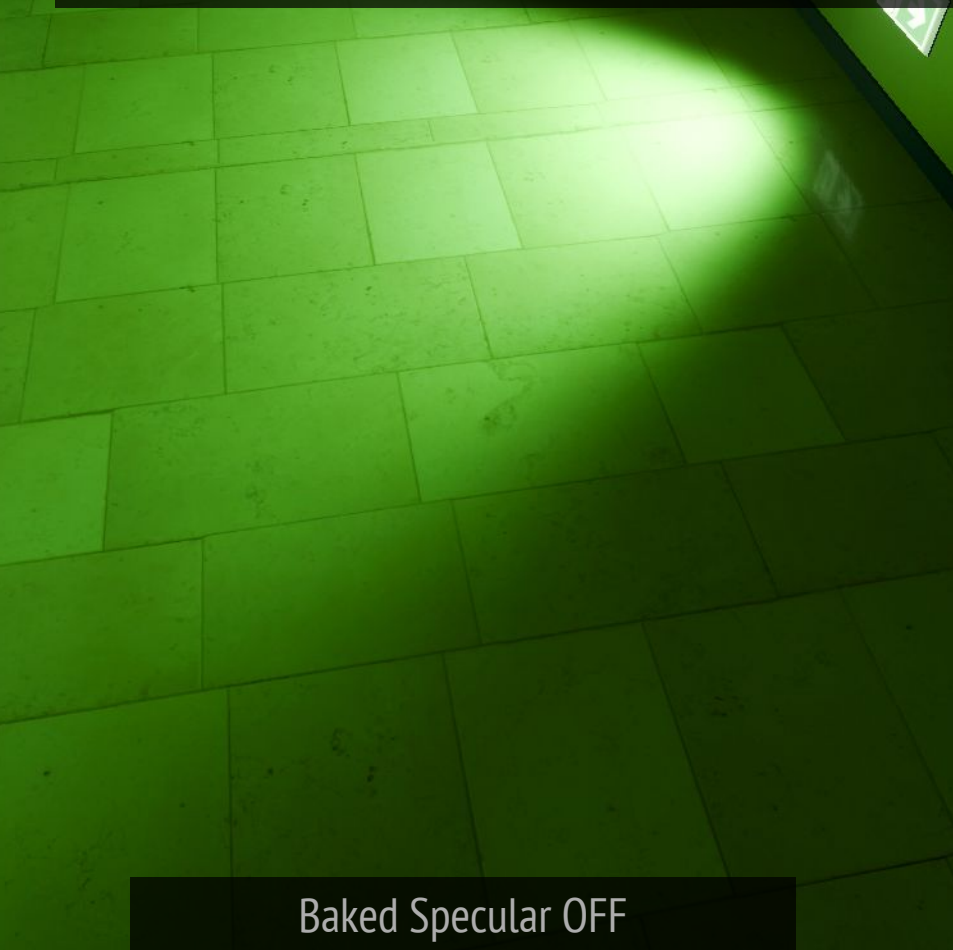


Baked Specular OFF



Baked Specular ON

Baked Specular : Example with a Spotlight where it's better to keep "Lightmapped Specular" OFF



Height map

Parallax Mapping:

- Used by Unity Standard
- Cheaper technique, but can appear flat or “glass-like” at shallow angles

Parallax Occlusion Mapping (POM):

- Used by Mochie, Filamented, Orels1...
- More expensive, but layers can occlude each other, making the surface appear more like real geometry

Example on the right : carpet texture

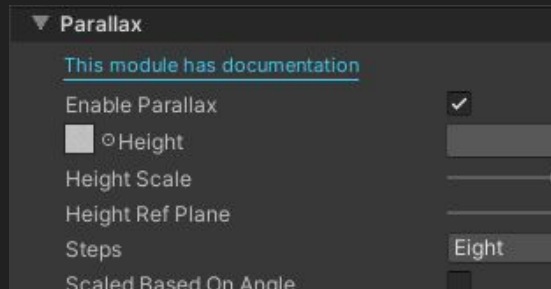


Height map - Step setting

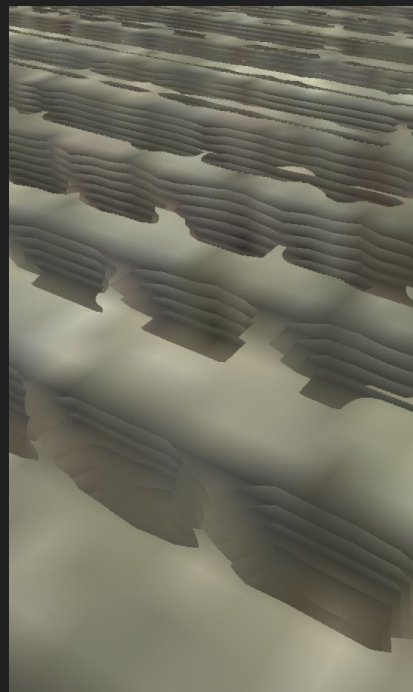
Higher step counts improve quality, but increase rendering cost

Some shaders do not expose the “Step” setting

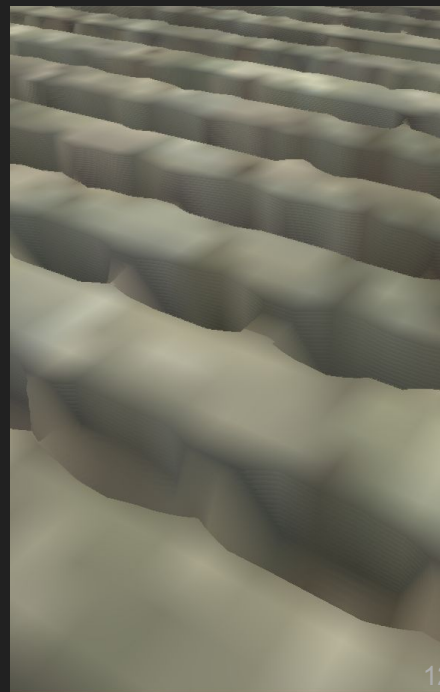
Example on the right : Orels shader



Step : 8



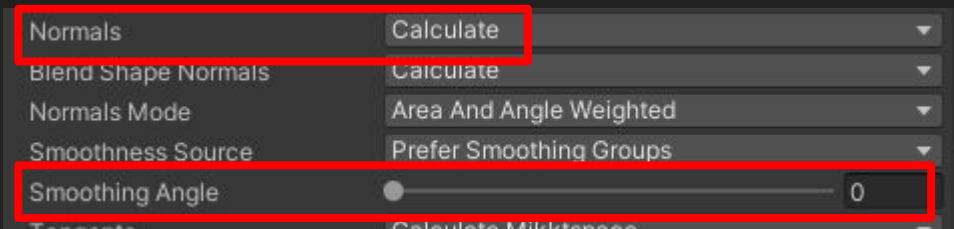
Steps : 32



Height map - Warping issue

If you get “Warping” artifacts when getting close to a parallax surface (first image), try recalculating the mesh normals : Import settings => “Smoothing Angle” to 0

You can also try a different shader, effect may vary.



Shader features - Screen Space Reflection (SSR)

- Computes real-time reflections in a shader
- Screen Space : Only reflect what's visible on the screen

Has a performance impact, it should not replace Reflection Probes, recommended as a toggleable option.

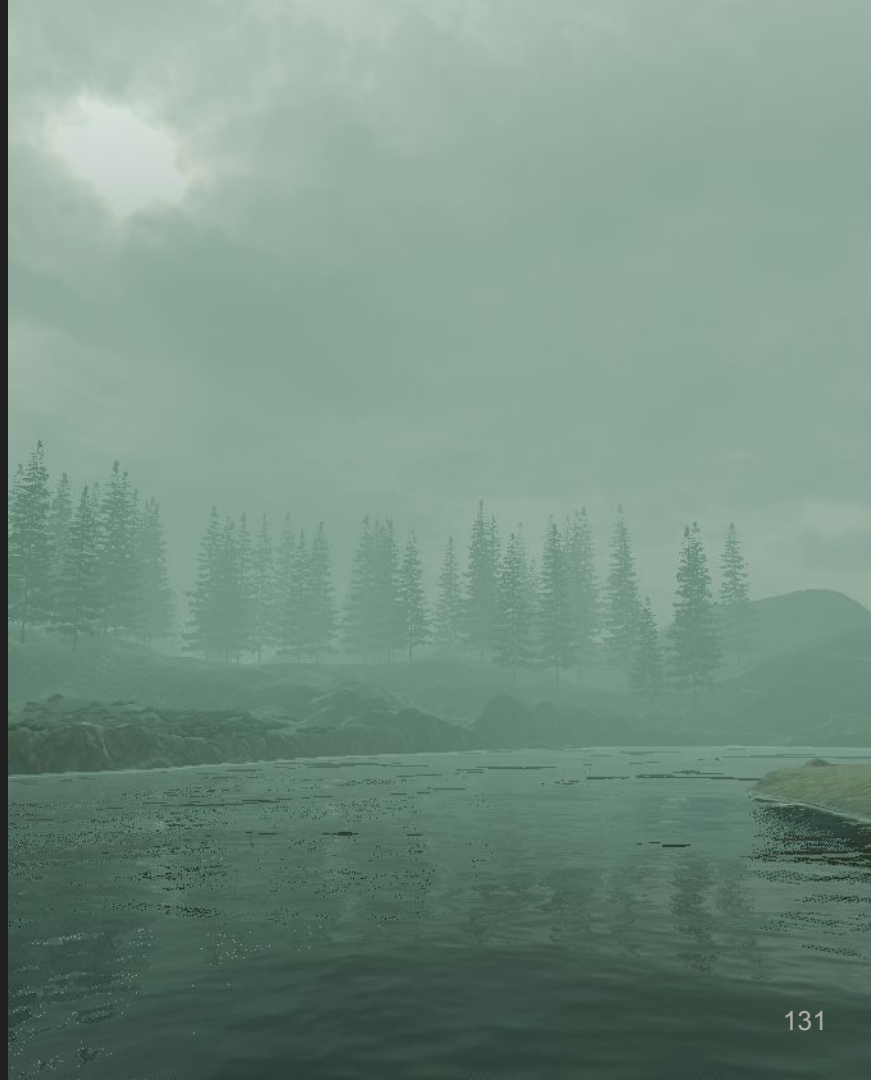


Fog

- Different ways to achieve fog:
 - Particle systems
 - Unity fog (Lighting tab)
 - Shaders (volumetric...)

Unity fog, easy to setup, but:

- Not applied to the skybox using the default skybox shader
- No height-based density (Alternative : Silent's SkyProbe Fog shader
<https://gitlab.com/s-ilent/skyprobe-fog>)



Volumetric fog

- + Ikeiwa's volumetric fog shader
- + Light volumes
- + Bakery Light Filter
- + Directional light

<https://github.com/Ikeiwa/Unity-Baked-Volumetrics>



Silent's Volumetric Fake Lights shader

<https://gitlab.com/s-ilent/fake-lights>

OFF



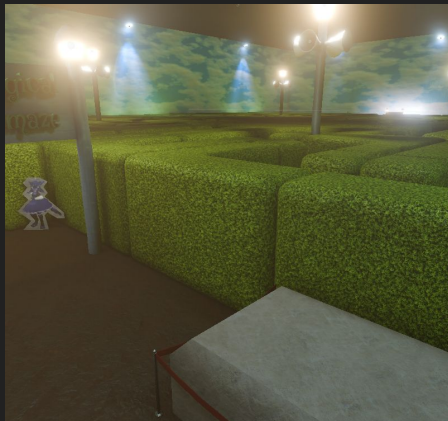
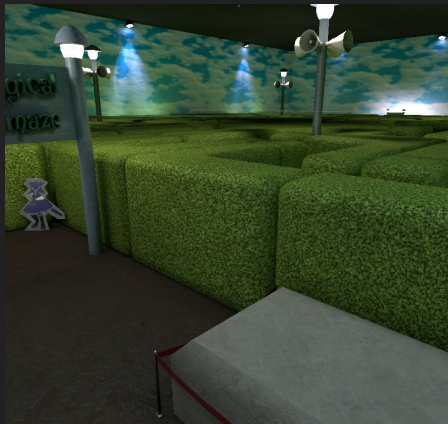
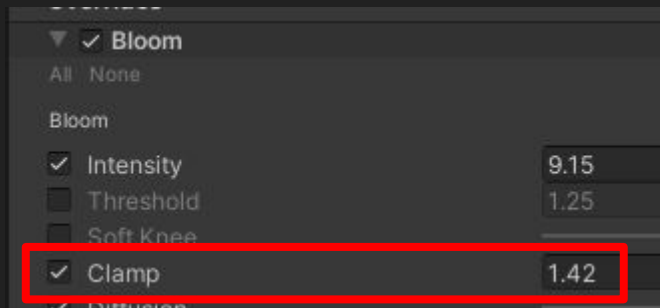
ON



Bloom

Avoid using bloom to simulate thick fog, as certain avatars have very strong emissive lights. Use bloom sparingly.

Clamping the bloom can reduce that issue



Silent Sunshine flare shader

Shader that renders a lens flare and crepuscular rays (sun rays) from the sun over the scene.

<https://gitlab.com/s-ilent/sunshine-flare>

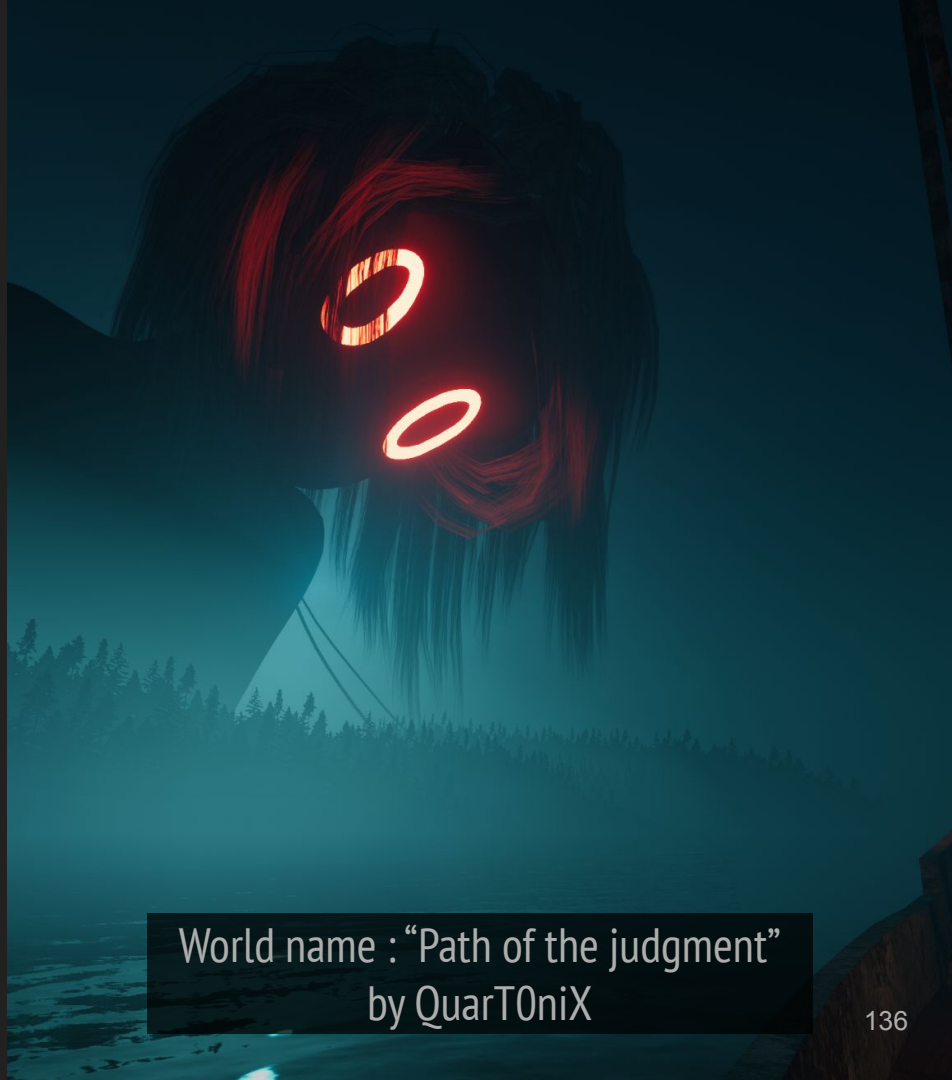


World name : "Matrix..."
by TontonDemon

Silent SkyProbe Fog

- Overlays a blurred reflection probe over the scene to simulate fog
- Supports height-based density
- Bright areas in the reflection probe also appear in the fog

<https://gitlab.com/s-ilent/skyprobe-fog>



World name : "Path of the judgment"
by QuarTOniX



Thanks !

Twitter : @MyroDev

Bsky : @myrodev.bsky.social

My personal recommendation to practice on lighting : Liminal spaces

Links

PBR shading guide : <https://www.patreon.com/posts/pbr-shaders-31084235>

Issues when baking your lightmaps?

- Unity Lightmapper
<https://discussions.unity.com/t/lightmapping-troubleshooting-guide/895352>
- Bakery <https://geom.io/bakery/wiki/index.php?title=Troubleshooting>
- Bakery documentation <https://geom.io/bakery/wiki/index.php?title=Manual>

Links

Videos about world creation and lighting :

- Furlity : Making immersive worlds <https://www.youtube.com/watch?v=aK2pwAWfVM4>
- Furlity : Building Better VRChat Worlds
<https://www.youtube.com/watch?v=uydb50pnbVO>
- Furlity : The Art of Light and Shading https://www.youtube.com/watch?v=th_6GtiyhZI